



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Душан Комадиновић

**Микросервисна платформа за каталог,
управљање личном библиотеком и
размену утисака о видео играма**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Душан Комадиновић	Број индекса:	SV65/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Микросервисна платформа за каталог, управљање личном библиотеком и размену утисака о видео играма

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quia ipsum suspendisse ultram dignissim parit. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur. Quis autem vel eum iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae consequatur, vel illum qui dolorem eum fugiat quo voluptas nulla pariatur.

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Душан Комадиновић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Микросервисна платформа за каталог, управљање личном библиотеком и размену утисака о видео играма
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	8/59/59/5/41/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Rust, микросервиси, софтверска архитектура, веб апликација, видео игре
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај рад описује дизајн и имплементацију микросервисне веб платформе која нуди јединствени каталог видео игара, уређивање личне библиотеке и размену утисака међу играчима. Пет доменских сервиса је иза API gateway-а, сваки са сопственим складиштем по обрасцу база података по сервису. Сервиси су пројектовани за независан развој, тестирање и пуштање у рад. Налози, каталог и библиотека су у PostgreSQL-у, кориснички садржај и метаподаци медија у MongoDB-у, а слике и видео у MinIO-у. Серверски део је у Rust-у (Axum), уз JWT (RS256) и контролу приступа према улози на gateway-у и сервисима. Nginx сервера Angular клијент, све се покреће преко Docker Compose-а и демонстрирано је кроз токове каталога, библиотеке и модерације.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Dušan Komadinović
Mentor, MN :	Igor Dejanović, PhD, full professor
Title, TI :	Microservice platform for cataloging, personal library management and sharing opinions on video games
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/59/59/5/41/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Rust, microservices, software architecture, web application, video games
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis presents a microservice web platform for a unified video game catalog, personal library management, and opinion sharing among players. Five domain services sit behind an API gateway, each with its own storage under the database-per-service pattern. Services are designed for independent development, testing, and deployment. Accounts, catalog, and library data reside in PostgreSQL; user content and media metadata in MongoDB; and images and video in MinIO. The backend is implemented in Rust (Axum), with JWT (RS256) and role-based access control at the gateway and across services. Nginx serves the Angular client, the platform runs via Docker Compose, and its use is demonstrated through catalog, library, and moderation flows.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	
President:	Petar Petrović, PhD, assoc. professor
Member:	Marko Marković, PhD, asist. professor
Member:	
Member, Mentor:	Igor Dejanović, PhD, full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	Теоријске основе	3
2.1	Стилови архитектуре	3
2.2	Микросервисни обрасци	3
2.3	Безбедносни концепти	4
2.4	Инфраструктурни концепти	4
2.5	Имплементациони концепти	5
2.6	<i>Rust</i> концепти	5
3	Захтеви и корисничке улоге	7
3.1	Корисничке улоге	7
3.2	Функционални захтеви	7
3.3	Нефункционални захтеви	9
4	Коришћене технологије	11
4.1	Програмски језик <i>Rust</i>	11
4.2	Складиштење података	11
4.3	Библиотеке серверског дела	11
4.4	<i>Angular</i>	12
4.5	<i>nginx</i>	13
4.6	<i>Docker</i>	13
5	Архитектура система	15
5.1	Микросервиси платформе	15
5.2	Организација кода	16
5.3	Изградња и покретање сервиса	17
6	Имплементација	19
6.1	Библиотечки сандуци <i>jwt-common</i> и <i>service-common</i>	19
6.2	Сервис за кориснике	20
6.3	Сервис за каталог	22
6.4	Сервис за библиотеку	24
6.5	Сервис за кориснички садржај	25
6.6	Сервис за мултимедију	26
6.7	<i>API gateway</i> сервис	27
6.8	Документација <i>API</i> -ја	29
6.9	Клијентска апликација	29
6.10	Покретање платформе	31
7	Пример коришћења	33
7.1	Припрема демонстрационих података	33
7.2	Кориснички токови	33
7.3	Административне операције	39
8	Закључак	43
	Списак слика	45
	Списак листинга	47
	Списак табела	49
	Списак коришћених скраћеница	51
	Списак коришћених појмова	53

Биографија	55
Литература	57

Током протеклих деценија индустрија видео игара је имала масован раст, прераставши у најдоминантнију у индустрији забаве [1]. Индустрија је 2018. године вредела 151,9 милијарди долара [1]. Платформе за дистрибуцију игара, као што су *Steam* и *GOG*, првенствено служе за куповину и играње игара [2], [3]. Њихови корисници имају могућност да пишу рецензије, оцењују игре и размењују мишљења кроз дискусије и коментаре [4], [5].

Постојеће платформе за дистрибуцију игара имају своја ограничења. Сав кориснички садржај се везује за конкретну платформу, а игре могу да се оцењују тек када се претходно купе. Последице тога су:

- доношење одлуке о куповини и игрању игара је отежано када се иста игра продаје на више платформи,
- будући играчи немају обједињен увид у мишљења других корисника и
- појединци који имају налоге на више платформи не виде све своје игре на једном месту.

На основу истакнутих проблема, постоји потреба за платформом која корисницима нуди: јединствени каталог игара, управљање личном библиотеком игара и размену утисака, независно од платформе на којој играју и поседују игре. Нека решења већ постоје: платформе *Backloggd* [6] и *RAWG* [7]. Обе имају широк каталог игара и нуде управљање личном библиотеком, оцењивање и коментарисање игара. Међутим, за постојеће платформе не постоји јавно доступна техничка документација за дубљу анализу архитектуре.

У оквиру овог рада је развијена платформа намењена свим играчима за откривање нових игара на основу препорука других и лакшу организацију сопствених колекција игара. Фокус развоја је био на архитектури и дизајну система, јер систем описане платформе треба да подржи велики број корисника и да притом буде модуларан, скалабилан и лако одржив. Серверски део платформе је имплементиран у програмском језику *Rust* применом микросервисне архитектуре [8] и одговарајућих микросервисних образаца [9].

Рад је подељен на осам поглавља. Неопходни појмови и концепти су дефинисани у поглављу 2. Захтеви система и корисничке улоге су набројани у поглављу 3. Коришћене технологије су наведене у поглављу 4. Архитектура система приказана је у поглављу 5. У поглављу 6 су приказани детаљи имплементације. У поглављу 7 су приказани конкретни примери коришћења платформе. Поглавље 8 доноси закључак и правце даљег развоја.

Глава 2

Теоријске основе

Ово поглавље уводи теоријске концепте неопходне за разумевање имплементације платформе. Концепти су груписани у шест целина. Најпре се наводе различити приступи изградње система у потпоглављу 2.1, након чега се представљају коришћени микросервисни обрасци у потпоглављу 2.2. Потом следи преглед примењених безбедносних концепата у потпоглављу 2.3, док су инфраструктурни и имплементациони концепти обухваћени потпоглављима 2.4 и 2.5. У последњем потпоглављу, 2.6, објашњени су кључни *Rust* концепти.

2.1 Стили архитектуре

Избор архитектуре система се своди на монолитну и микросервисну. У наставку овог потпоглавља су описане обе архитектуре, а у табели 1 су истакнуте кључне разлике између њих.

2.1.1 Монолитна архитектура

Код монолитне архитектуре систем је структуриран као једна компонента која се цела пушта у рад и која користи заједничку базу података, а притом обухвата све поддомене система [8], [9]. Предност је што су све интеракције локалне и не захтевају мрежну комуникацију, па су операције ефикасне [9]. Недостаци расту са величином система и тима: код постаје тежак за одржавање и изолованост рада тимова опада због дељеног кода [9].

2.1.2 Микросервисна архитектура

Микросервисна архитектура дели систем на скуп сервиса који се независно пуштају у рад, при чему сваки сервис обухвата један или више поддомена [8], [9]. Овакав приступ омогућава тимовима независан развој, тестирање и пуштање у рад сваког сервиса [8], [9]. Цена ове поделе је сложеност дистрибуираних операција које обухватају више сервиса [9].

Табела 1: Поређење монолитне и микросервисне архитектуре, према Ричардсону [9]

Критеријум	Монолит	Микросервиси
Структура компонената	једна компонента која обухвата све поддомене	скуп мањих, независних сервиса - сваки за групу поддомена
Изолованост рада тимова	тимови деле заједнички код	независан развој по сервису
Брзина пуштања у рад	систем се пушта у рад као целина	сваки сервис се пушта у рад независно од других
Технолошки стек	јединствен за цео систем	независан по сервису, прилагођен потребама

2.2 Микросервисни обрасци

2.2.1 API gateway

API gateway је образац по коме један сервис постаје јединствена улазна тачка система [8], [9]. Одговоран је за рутирање захтева ка одговарајућим сервисима, служећи као *проху*, а може да обавља и аутентификацију корисника [8], [9]. Ова централизација уводи додатну тачку сложености и мрежни скок, али је тај скок често занемарљив [9].

2.2.2 База података по сервису

База података по сервису (енгл. *database per service*) је образац по коме су трајни подаци сваког микросервиса приватни за тај сервис и доступни искључиво преко API-ја сервиса [9]. Приватност базе се може остварити на више начина: од приватних табела и засебне шеме базе по сервису до потпуно засебног сервера базе података [9]. Овај образац обезбеђује лабаву спрегу и слободу избора технологије базе по сервису, али отежава имплементацију трансакција и упита који захтевају податке из више сервиса [9].

2.3 Безбедносни концепти

2.3.1 JWT

JWT (енгл. *JSON Web Token*) је стандард за безбедно преношење скупа информација (енгл. *claims*) преко мреже између две стране, најчешће у сврху аутентификације корисника [10]. JWT омогућава реализацију *stateless* сесија: сервер не мора да чува стање корисничке сесије [10]. Интегритет и аутентичност JWT-а обезбеђују се његовим криптографским потписивањем, при чему се могу користити различити алгоритми. У имплементацији је коришћен RS256 (енгл. *RSA Signature with SHA-256*), асиметрични криптографски алгоритам.

2.3.2 Access и refresh токени

Аутентификација помоћу JWT најчешће подразумева пар токена: *access* и *refresh*, које корисник добија приликом пријаве. *Access* токен је краткотрајан (важи свега неколико минута) и шаље се уз сваки захтев до истека, при чему га сервер може самостално верификовати путем потписа [10]. *Refresh* токен је дуготрајан (важи више дана) и користи се искључиво за издавање новог пара токена када *access* токен истекне [10].

Уобичајена пракса је да се при свакој употреби *refresh* токена стари прогласи неважећим и да се изда нов пар (*token rotation*), при чему сервер мора да чува издате *refresh* токене како би могао да их повуче [11]. Овакав приступ омогућава кориснику да остане пријављен док год редовно користи систем, а истовремено ограничава штету у случају крађе: украден *access* токен брзо истиче, што је прихватљив компромис, док украден *refresh* токен представља озбиљан ризик и мора бити максимално заштићен [11].

2.3.3 Zero trust

Zero trust је безбедносни образац изграђен на принципу „никада не веруј, увек провери“ (енгл. *never trust, always verify*) [12]. По њему, ниједан захтев није поуздан без претходне провере, без обзира на то да ли долази изван или унутар мреже [12].

2.4 Инфраструктурни концепти

2.4.1 Контејнеризација

Контејнеризација (енгл. *containerization*) омогућава паковање апликације заједно са свим што јој је потребно за рад у изоловане јединице - контејнере, који се покрећу конзистентно у било ком окружењу [13]. Изолација се постиже на нивоу оперативног система [13].

2.4.2 Reverse proxy

Reverse proxy је посредник између клијента и сервера који првенствено служи да прослеђује захтев у име клијента и сакрије унутрашњу структуру система [14].

2.4.3 Објектно складиште

Објектно складиште (енгл. *object storage*) чува податке у виду објеката - произвољних бинарних података попут слика, аудио и видео записа [15]. Објекти се смештају у именоване *bucket*-е [15].

2.4.4 OpenAPI

OpenAPI је стандард за описивање интерфејса HTTP API-ја који омогућава људима и софтверским алатима да разумеју његове могућности без увида у изворни код имплементације [16]. Опис је JSON или YAML

документ који за сваки *endpoint* дефинише доступне операције, очекиване параметре и могуће одговоре, што алатима омогућава да из њега аутоматски генеришу документацију [16].

2.5 Имплементациони концепти

2.5.1 Оптимистично закључавање

Оптимистично закључавање (енгл. *optimistic locking*) је техника очувања интегритета података код конкурентних операција [17]. Заснива се на постојању бројчане колоне: верзија у шеми базе података, која се увећава при свакој измени податка. Операција пролази ако се верзија која је пристигла из захтева поклапа са актуелном верзијом податка у бази, у супротном се операција одбија [17].

2.5.2 *Package by feature*

Package by feature је модел организовања кода у коме су сви фајлови везани за једну функционалност груписани у исти фолдер [18]. Овакво груписање омогућава да структура кода одражава пословни домен апликације, а не техничке слојеве, што олакшава проналазак и измену кода везаног за одређени случај коришћења [18].

2.6 Rust концепти

2.6.1 Власништво

Власништво (енгл. *ownership*) је скуп правила помоћу којих Rust у време компајлирања управља меморијом, без коришћења *garbage collector*-а [19]. Кршење било ког правила спречава компајлирање програма [19]. Правила су:

- свака вредност има свог власника (енгл. *owner*),
- вредност може да има само једног власника у сваком тренутку и
- вредност се деалоцира (енгл. *drop*) чим власник изађе из опсега важења (енгл. *scope*) [19].

2.6.2 Позајмљивање

Позајмљивање (енгл. *borrowing*) омогућава да се вредности приступи преко референце (&), без преузимања власништва, при чему референца мора увек да показује на важећу вредност [19]. У сваком тренутку дозвољена је или једна променљива референца (декларисана са кључном речју *mut*) или произвољан број непроменљивих референци над истом вредношћу, никада обе врсте истовремено [19].

2.6.3 *Trait*

Trait је концепт сличан интерфејсу у другим језицима који дефинише скуп метода које тип мора да имплементира [19]. Како Rust нема механизме наслеђивања, понашање се дели искључиво имплементацијом *trait*-ова.

2.6.4 Сандуци

Сандук (енгл. *crate*) је јединица компајлирања [19]. Сандук може бити бинарни (енгл. *binary*), који се компајлира у извршни фајл, или библиотечки (енгл. *library*), који излаже функционалности другим сандуцима [19]. У оквиру пројекта, више сандука може да се организује у заједнички *workspace*.

Глава 3

Захтеви и корисничке улоге

Ово поглавље наводи корисничке улоге, функционалне и нефункционалне захтеве платформе, редом у потпоглављима 3.1, 3.2 и 3.3. Преглед приступа функционалностима по улогама сажет је у табели 2.

3.1 Корисничке улоге

Платформа дефинише четири корисничке улоге са различитим нивоима приступа функционалностима, при чему свака виша улога има привилегије свих нижих:

- нерегистровани корисник (гост),
- (обичан) корисник,
- модератор и
- администратор.

Табела 2: Приступ функционалностима по улогама

Функционалност	Гост	Корисник	Модератор	Администратор
Преглед каталога игара	✓	✓	✓	✓
Уређивање каталога игара	–	–	–	✓
Управљање личном библиотеком	–	✓	✓	✓
Оцене и коментари	–	✓	✓	✓
Подношење пријава о непримереном садржају	–	✓	✓	✓
Преглед профила и библиотека корисника	✓	✓	✓	✓
Регистрација	✓	–	–	–
Аутентификација и управљање личним налогом	–	✓	✓	✓
Управљање мултимедијалним садржајем игара	–	–	–	✓
Модерација непримереног садржаја	–	–	✓	✓
Управљање налозима корисника	–	–	–	✓

3.2 Функционални захтеви

3.2.1 Преглед каталога игара

Сви корисници могу да претражују игре по називу и филтрирају их по платформама (*Windows*, *Playstation 5*, *Xbox*, итд.), жанровима и ознакама, уз сортирање и пагинацију.

Страница са детаљима игре приказује:

- основне податке (назив, опис, датум изласка, веб сајт),
- пратеће ентитете (програмере, издаваче, жанрове, платформе и ознаке),
- слике и видео снимке (насловна слика, снимци екрана, трејлер игре),
- оцене са рецензијама (могуће филтрирање, пагинација) и
- коментаре (пагинација).

Избором програмера или издавача на страници игре могуће је прегледати остале игре од истог програмера, односно издавача.

3.2.2 Уређивање каталога игара

Администратори управљају каталогом игара. Нова игра се након уноса основних података чува као нацрт, видљива само администраторима. Након тога могуће је додати мултимедијални садржај. Када су подаци комплетни, администратори могу да објаве игру, чиме постаје видљива свим корисницима, или да је врате у нацрт. У сваком тренутку могу да ажурирају податке о игри, а могу да је обришу само док је као нацрт.

Поред самих игара, администратори одржавају и пратеће податке каталога:

- програмере,
- издаваче,
- жанрове,
- платформе и
- ознаке.

3.2.3 Управљање личном библиотеком игара

Регистровани корисници додају и уклањају игре из личне библиотеке и групишу их према статусу:

- „у власништву“,
- „тренутно се игра“,
- „листа жеља“,
- „пређена“ и
- „одустао“.

3.2.4 Оцене и коментари

Регистровани корисници могу оцене игру и да напишу рецензију за било коју игру из каталога (обавезна оцена уз опциони текст). Накнадно могу да је измене или обришу. Оцена може бити:

- “топла препорука”,
- “препоручује се”,
- “прихватљиво” и
- “не препоручује се”.

Регистровани корисници такође могу да коментаришу игру и оцене других корисника, уз могућност измене и брисања сопствених коментара.

3.2.5 Подношење пријава о непримереном садржају

Регистровани корисници могу да поднесу пријаву о непримереном коментару или оцени, уз образложење. Није могуће пријавити сопствени садржај.

3.2.6 Преглед профила и библиотека корисника

Сви корисници могу да виде профиле других корисника и њихове библиотеке игара.

3.2.7 Регистрација

Нерегистровани корисници могу да се региструју креирањем налога, уносом минимално корисничког имена, имејл адресе и лозинке.

3.2.8 Аутентификација и управљање личним налогом

Регистровани корисници могу да се пријаве и приступе функционалностима у складу са својом улогом. У било ком тренутку могу да се одјаве. У оквиру управљања налогом, регистровани корисници могу да уређују личне податке, мењају лозинку и деактивирају свој налог.

3.2.9 Управљање мултимедијалним садржајем игара

Администратори постављају и бришу слике и видео записе везане за игру. Садржај постаје видљив осталим корисницима тек након објаве игре.

3.2.10 Модерација непримереног садржаја

Модератори и администратори имају увид у пријављене коментаре и оцене. Пријаву могу да уваже, при чему се пријављени садржај уклања, или да је одбаце.

3.2.11 Управљање налозима корисника

Администратори мењају улоге корисника, додељујући им или одузимајући привилегије. Такође могу трајно да блокирају налог корисника и тиме му онемогуће даљи приступ налогу.

3.3 Нефункционални захтеви

3.3.1 Модуларност и независан развој

Систем је подељен на домене тако да се појединачне функционалности могу развијати и проширивати без утицаја на рад остатка система.

3.3.2 Скалабилност и перформансе

Систем је пројектован да очува прихватљив одзив претраге и филтрирања каталога при расту броја корисника, игара, коментара и оцена.

3.3.3 Безбедност и контрола приступа

Систем мора да спречи неовлашћен приступ функционалностима и подацима у зависности од улоге корисника и власништва над подацима.

Ово поглавље образлаже избор технологија платформе. Серверски део покривају програмски језик *Rust* (потпоглавље 4.1), системи за складиштење података (потпоглавље 4.2) и библиотеке (потпоглавље 4.3). Клијентски део покривају радни оквир *Angular* (потпоглавље 4.4) и веб сервер *nginx* (потпоглавље 4.5), а покретање и пуштање у рад платформе покрива *Docker* (потпоглавље 4.6).

4.1 Програмски језик *Rust*

Rust је системски програмски језик чије власништво и позајмљивање обезбеђују безбедност меморије у време компајлирања без *garbage collector*-а [19]. Тиме се безбедност при конкурентном опслуживању великог броја захтева проверава у време компајлирања. Строги систем типова захтева експлицитну обраду грешака употребом типова *Result* и *Option*, па се велики део грешака открива пре покретања програма [19]. Организација кода у сандуке у оквиру *workspace*-а омогућава независан развој, тестирање и изградњу појединачних делова система, што одговара захтеву за модуларношћу платформе из поглавља 3.3.1.

4.2 Складиштење података

4.2.1 *PostgreSQL*

PostgreSQL је одабран за структуриране податке са јасним међусобним везама и потребом за трансакцијама [20]. За убрзање претраге користе се секундарни индекси и GIN индекс (уопштени инвертовани индекс, енгл. *generalized inverted index*) за текстуалну *ILIKE* претрагу по називу помоћу екстензије *pg_trgm*, при чему је триграм низ од три узастопна карактера [21], [22]. Оператор *ILIKE* врши поређење без разликовања величине слова (енгл. *case insensitive*) [22].

4.2.2 *MongoDB*

MongoDB је одабран за документе променљиве структуре [23]. Подаци који припадају заједно: угнеђени садржаји, опциона поља и везе ка другим документима стају у исти документ, док би у релационом моделу такви подаци захтевали додатне табеле. Претрага се убрзава индексима, укључујући сложене индексе преко више поља који истовремено подржавају филтрирање и сортирање [23].

4.2.3 *MinIO*

MinIO [15] је одабран као S3-компатибилно објектно складиште за слике и видео записе. За разлику од ручног чувања на систему датотека сервера, где апликација сама брине о распореду, путањама и трајности датотека, чување је препуштено складишту које нуди приступ преко мрежног API-ја [15].

4.3 Библиотеке серверског дела

4.3.1 *axum*, *tokio* и *tower-http*

Сандук *axum* [24] је одабран за изградњу HTTP интерфејса. Захтеви се преко *Router* структуре усмеравају ка функцијама за обраду захтева (енгл. *handlers*), при чему се делови захтева преузимају екстракторима (енгл. *extractors*) као што су *State*, *Json*, *Path* и *Query*, а одговори се враћају типовима који имплементирају *IntoResponse* [24]. Тиме се облик захтева и одговора проверава типовима у време компајлирања. Извршавање се ослања на асинхронно извршно окружење *tokio* [25].

Додатно понашање захтева и одговора обликује се слојевима сандука *tower-http* [26]:

- праћење захтева,
- дељење ресурса између различитих порекла (енгл. *Cross-Origin Resource Sharing - CORS*),

- ограничење трајања обраде и
- нормализација путање.

Ограничење величине тела захтева обезбеђује *axum*.

4.3.2 *reqwest*

Сандук *reqwest* пружа HTTP клијент високог нивоа [27]. Користи се за слање захтева и обраду одговора, уз подршку за токове и ограничење времена одзива [27].

4.3.3 *serde* и *validator*

Сандуци *serde* и *validator* чине заједничку основу за обраду улазних и излазних података [28], [29]. Сандуком *serde* се структуре захтева и одговора серијализују и десеријализују, укључујући JSON приказ преко HTTP-а, уз подршку за типове датума, времена и јединствених идентификатора. Сандуком *validator* се декларативно описују ограничења над пољима улазних структура, па се неисправни захтеви одбијају пре даље обраде [29].

4.3.4 *sqlx*

Приступ бази *PostgreSQL* омогућава сандук *sqlx* [30]. Упити се проверавају према шеми базе у време компајлирања, па се грешке у синтакси и типовима колона откривају пре покретања програма. Сви упити су параметризовани, чиме се спречава *SQL injection* напад [30]. Подржава асинхрони *connection pool*, аутоматско мапирање типова датума, времена и јединствених идентификатора, као и верзионисане миграције шеме [30].

4.3.5 *mongodb*

Приступ бази *MongoDB* омогућава званични сандук *mongodb* [31]. Подржава агрегационе пајплајне (енгл. *pipelines*) за груписање вредности из више докумената и израчунавање једног резултата, као и програмско креирање индекса [31], [32].

4.3.6 *minio*

Приступ *MinIO* објектном складишту омогућава званични сандук *minio* [33]. Подржане су основне операције складиштења и брисања објеката, при чему се за читање издају унапред потписани (енгл. *presigned*) URL-ови са ограниченим роком важења, што омогућава директан приступ складишту [33].

4.3.7 *utoipa*, *utoipa-axum* и *utoipa-swagger-ui*

Сандуци *utoipa* и *utoipa-swagger-ui* служе за описивање REST интерфејса према *OpenAPI* стандарду [34]. Помоћу макроа `#[utoipa::path]` над функцијама за обраду захтева и структурама у време компајлирања настаје *OpenAPI* спецификација, која се приказује кроз интерактивни кориснички интерфејс. Тиме се *endpoint*-и могу прегледати и испробати пре израде клијентске апликације [34]. Сандук *utoipa-axum* нуди омотач *OpenApiRouter* око *Router* структуре из сандука *axum*, који уз регистрацију рута прикупља и њихову спецификацију [35].

4.3.8 *argon2*

Сандук *argon2* служи за хеширање лозинки пре чувања у базу [36]. Уз сваку лозинку се чува јединствена со (енгл. *salt*), па исте лозинке дају различите хешеве [37].

4.4 *Angular*

Angular [38] је одабрани радни оквир за клијентски део платформе, због свог компонентног система и ослањања на *TypeScript* [39] (надскуп језика *JavaScript* са статичким типовима), па се грешке у типовима откривају у време компајлирања уместо при извршавању у прегледачу. Примењене су препоручене праксе организације компонената: свака страница је засебна компонента, а заједнички делови су издвојени у компоненте ради поновне искористивости, што олакшава одржавање.

За изглед апликације коришћене су библиотеке *Angular Material* [40] и *Bootstrap* [41]. За стил елемената страница су примарно коришћене компоненте из *Angular Material*, док је *Bootstrap* служио само за распоред и величину елемената. При изради се водило рачуна о прилагођавању изгледа величини екрана уређаја, тзв. *responsive design*. За управљање зависностима, изградњу и покретање коришћен је алат *Bun* [42].

4.5 *nginx*

nginx је одабран да сервира *Angular* клијентску апликацију, а такође служи као *reverse proxy* [14]. Може да сервира статичке датотеке уз подршку за рутирање на клијенту и да преостале захтеве прослеђује серверском делу, а кеширањем, *gzip* компресијом и сигурносним заглављима се побољшавају сервирање и сигурност [14].

4.6 *Docker*

За покретање платформе користе се *Docker* и *Docker Compose*, у складу са контејнеризацијом [43]. *Docker* слика (енгл. *image*) се описује датотеком *Dockerfile* као декларативним рецептом са корацима попут преузимања основне слике, копирања датотека и навођења команде покретања [44], при чему се слојеви кеширају ради брже изградње [45]. Више контејнера се заједно описује датотеком *compose.yml* кроз сервисе, слике, мреже, волумене и зависности [46]. За различита окружења користи се више *compose.yml* датотека [47].

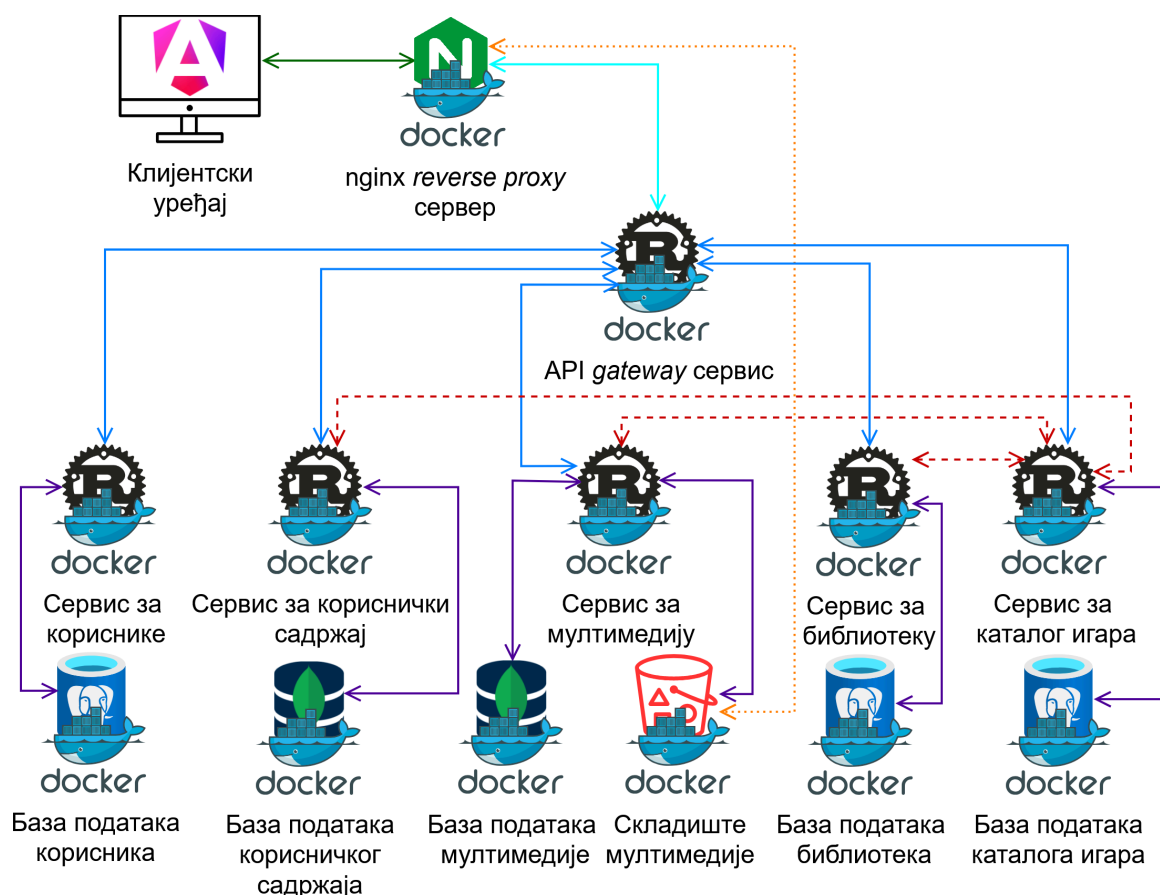
Ово поглавље приказује архитектуру платформе. Микросервиси и њихове одговорности дати су у потпоглављу 5.1. Организација кода описана је у потпоглављу 5.2, а изградња и покретање сервиса у потпоглављу 5.3.

5.1 Микросервиси платформе

На основу упоредног приказа стилова архитектуре из поглавља 2.1 и нефункционалних захтева из поглавља 3.3, за платформу је изабрана микросервисна архитектура. Преглед шест сервиса платформе дат је у табели 3, а сервиси су формиран на основу функционалних захтева из поглавља 3.2. Сви сервиси сем *API gateway* сервиса имају сопствени модел и базу података, по обрасцу база података по сервису [9]. Сви сервиси излажу REST интерфејс описан *OpenAPI* спецификацијом [16]. Архитектура је илустрована на слици 1.

Табела 3: Микросервиси платформе

Сервис	Одговорности	База података
Сервис за кориснике	Регистрација, пријава, издавање и управљање JWT, управљање налозима и улогама	<i>PostgreSQL</i>
Сервис за каталог игара	Основни подаци о играма, пратећи ентитети, претрага и преглед	<i>PostgreSQL</i>
Сервис за библиотеку	Веза корисник-игра	<i>PostgreSQL</i>
Сервис за мултимедију	Складиштење и обрада слика и видео снимака игара	<i>MongoDB</i> (метаподаци) + <i>MinIO</i> (фајлови)
Сервис за кориснички садржај	Оцене, коментари и модерација непримереног садржаја	<i>MongoDB</i>
API gateway сервис [9]	Рутирање захтева, валидација JWT, контрола приступа према улози корисника	-



Слика 1: Архитектура платформе са током комуникације између компоненти. Главни ток је представљен пуним линијама.

5.2 Организација кода

Пројекат је организован као *Rust workspace* (листинг 1) у коме је сваки сервис посебан бинарни сандук, док су `jwt-common` и `service-common` библиотечки сандуци. Припадници *workspace*-а и зависности пројекта су декларисани у коренском `Cargo.toml` фајлу, а верзије зависности закључане `Cargo.lock` фајлом.

```
playlog-backend/
├── services/
│   ├── api-gateway/
│   ├── catalogue-service/
│   ├── library-service/
│   ├── multimedia-service/
│   ├── review-service/
│   └── user-service/
├── shared/
│   ├── jwt-common/
│   └── service-common/
├── Cargo.toml
└── Cargo.lock
```

Листинг 1: Организација *Rust* пројекта

Унутар сваког сервиса код је организован по моделу *package by feature* [18]. Фолдер `games` сервиса за каталог (листинг 2) је типичан пример: обрада захтева, пословна логика и приступ подацима раздвојени су у `handler.rs`, `service.rs` и `repository.rs` фајловима и пратећи модели у фајловима `model.rs`, `dto.rs` и `error.rs`. Сви репозиторијуми, као и складиште објеката, дефинисани су *trait*-овима, па сервисни слој не зависи од њихових конкретних имплементација.

```

catalogue-service/src/games/
├── dto.rs
├── error.rs
├── handler.rs
├── model.rs
├── repository.rs
└── service.rs

```

Листинг 2: Пример *package by feature* организације

Библиотека, као најмањи сервис, приказује комплетну структуру фајлова (листинг 3). Улаз у сервис (`main.rs`) учитава конфигурацију, успоставља везу са базом и покреће апликацију; `config.rs` дефинише параметре уčitane из окружења, а `app.rs` спаја дељено стање и рутер у апликацију коју `main.rs` покреће. `docs.rs` дефинише структуру за генерисање *OpenAPI* спецификације сервиса.

Дељено стање и подаци о пријављеном кориснику (*claims*) стижу *handler*-има кроз екстракторе `State` и `Extension`, како је описано у поглављу 4.3.1.

Преостале датотеке подржавају изградњу и повезивање сервиса у платформу: `Cargo.toml` наводи зависности које сервис увози из *workspace*-а, `Dockerfile` гради слику сервиса, а `compose.yaml`, `Makefile` и `.env` датотеке га повезују у платформу. `migrations/` чува верзионисане измене шеме базе које се примењују при покретању, а `.sqlx/` кеширане упите за проверу у време компајлирања без везе са базом.

```

library-service/
├── .sqlx/
├── migrations/
├── src/
│   ├── app.rs
│   ├── config.rs
│   ├── docs.rs
│   ├── dto.rs
│   ├── error.rs
│   ├── handler.rs
│   ├── main.rs
│   ├── model.rs
│   ├── repository.rs
│   └── service.rs
├── .env
├── .env.production
├── Cargo.toml
├── compose.yaml
├── Dockerfile
└── Makefile

```

Листинг 3: Комплетан склоп сервиса

5.3 Изградња и покретање сервиса

Сви сервиси деле исти редослед покретања (листинг 4). Покретање почиње читањем `.env` датотеке, успостављањем праћења дијагностика и учитавањем конфигурације. Затим следе повезивање на складишта података и припрема шеме. На крају се повезани репозиторијуми и сервиси (структуре из слоја пословне логике) деле кроз заједничко стање, гради рутер апликације и покреће опслуживање.

Корак припреме шеме разликује се по типу базе: сервиси који користе *PostgreSQL* примењују верзионисане миграције, док они са *MongoDB* програмски иницијализују колекције и индексе. Поједини сервиси имају и додатне кораке: сервис за мултимедију иницијализује *MinIO*, а сервис за кориснике проверава постојање налога иницијалног администратора и покреће периодични задатак за брисање истеклих *refresh* токена.

API *gateway* сервис, како нема базу, иницијализује само дељени HTTP *proxy* клијент за комуникацију са осталим сервисима.

```
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    dotenvy::dotenv().ok();
    init_tracing(env!("CARGO_CRATE_NAME"));
    let env = load_from_environment()?;
    let pool = init_sqlx_db(&env.database_url).await?;
    sqlx::migrate!().run(&pool).await.context("Migrations failed")?;
    // ... инстанцирање репозиторијума и сервиса ...
    let state = Arc::new(AppState { config: env.app_config, /* ... */ });
    let app = build_app(state);
    let server_address = SocketAddr::from([0, 0, 0, 0], 3002);
    let listener = tokio::net::TcpListener::bind(&server_address).await?;
    tracing::info!("Server started. View docs at http://{}/docs", server_address);
    axum::serve(listener, app).with_graceful_shutdown(shutdown_signal()).await?;
    Ok(())
}
```

Листинг 4: Заједнички редослед покретања (скраћено из `library-service/src/main.rs`)

Читање конфигурације обједињује параметре учитане из окружења (у случају сервиса за библиотеку - јавни RSA кључ и URL до сервиса за каталог) у `AppConfig`. Заједничко стање `AppState`, изграђено у `main.rs`, обједињује конфигурацију и инстанциране сервисе, а до *handler*-а стиже управо кроз екстрактор `State`.

Апликацију гради функција `build_app`: над `OpenApiRouter`-ом се региструје рута за проверу здравља (енгл. *healthcheck*), угнежђује рутер сваке целине и примењују слојеви из поглавља 4.3.1. Помоћна функција `finalize_router` из `service_common` затим угњежђује изграђени рутер под путању `/api` и додаје интерактивну документацију на `/docs`, чиме настаје коначна апликација коју `main.rs` покреће.

Ово поглавље приказује детаље имплементације целе платформе, од заједничких сандука до покретања. Прво су издвојени заједнички сандуци у потпоглављу 6.1. Сервис за кориснике, сервис за каталог и сервис за библиотеку обрађени су у потпоглављима 6.2, 6.3 и 6.4. Сервис за кориснички садржај, сервис за мултимедију и API gateway обрађени су у потпоглављима 6.5, 6.6 и 6.7. Документација API-ја дата је у потпоглављу 6.8. Затим следи клијентска апликација у потпоглављу 6.9. На крају је дато покретање платформе у потпоглављу 6.10.

6.1 Библиотечки сандуци `jwt-common` и `service-common`

Заједничка основа издвојена је у два библиотечка сандука како се провера токена, облик грешака и приступ бази не би понављали у сваком сервису.

6.1.1 Сандук `jwt-common`

Набројив тип корисничких улога (Role) има три вредности: корисник, модератор и администратор. Скуп информација за JWT (Claims) носи просте типове: идентификатор, рокове и издаваоца, уз опционе улогу, корисничко име и имејл адресу - *access* попуњава све, а *refresh* само идентификатор. Конкретизоване структуре RefreshTokenClaims и AuthClaims настају провереним претварањем (TryFrom) из Claims и носе доменске типове: Uuid идентификатор, уз обавезан Role у AuthClaims.

Проверу токена спроводи ланац *middleware*-а (листинг 5). Конфигурација JwtConfig, која носи учитани јавни RSA кључ, прослеђује се као *axum* стање. *auth* издваја токен из заглавља, декодира га и убацује *claims* у екстензије захтева. Омотачи *require_user*, *require_moderator* и *require_admin* око *require_roles* прослеђују дозвољене улоге заједничкој провери, која чита *claims* и одбија недозвољене.

```
pub async fn auth(
    State(config): State<JwtConfig>,
    mut req: Request,
    next: Next,
) -> Result<Response> {
    let token = extract_bearer_token(req.headers())
        .map_err(|err| JwtError::InvalidAuthorizationHeader(err.to_string()))?;
    let claims = decode_token(&token, &config.public_key)?;
    let auth_claims = AuthClaims::try_from(claims)?;
    req.extensions_mut().insert(auth_claims);
    Ok(next.run(req).await)
}

async fn require_roles(req: Request, next: Next, allowed_roles: &[Role]) -> Result<Response>
{
    let claims = req.extensions().get::<AuthClaims>().ok_or(JwtError::MissingClaims)?;
    if !allowed_roles.contains(&claims.role) {
        Err(JwtError::Forbidden)
    } else {
        Ok(next.run(req).await)
    }
}
```

Листинг 5: Провера токена и улога (`jwt-common/src/middleware.rs`)

Варијанта `auth_optional` никада не одбија: *claims* убацује само при успешном декодирању токена, за јавне руте са додацима за пријављене.

Грешке `JwtError` пресликавају се у статусе: неисправности заглавља, токена и *claims* у 401, недовољна улога у 403, а неисправан кључ у 500.

6.1.2 Сандук *service-common*

Грешке обједињује `ApiError` (листинг 6), сачињену од статусног кода и списка порука, који се враћају као JSON низ. Свака целина у сервисима понавља исти образац: дефинише сопствени тип грешке и алијас типа `Result<T>` везан за њега, уз претварање у `ApiError` преко `From`, па се грешке помоћу оператора `?` саме прослеђују *handler*-има који враћају `ApiResponse`. Изузетак је *gateway*, чија грешка има сопствени одговор.

```
#[derive(Debug)]
pub struct ApiError {
    pub code: StatusCode,
    pub errors: Vec<String>,
}

pub type Result<T> = std::result::Result<T, ApiError>;
```

Листинг 6: Заједничка грешка и алијас типа `Result` (*service-common/src/error.rs*)

Пагинација се постиже са `PagedResponse<T: ToSchema>` структуром: резултат претраге уз укупан број ставки, број страница, тренутна страница и граница.

Проверу постојања игре сервисима за библиотеку, мултимедију и кориснички садржај нуди `CatalogueClient`: позив ка сервису за каталог игара мапира се у успех, непостојање (`NotFound`) или недоступност (`Unavailable`). Клијент се гради са роком од 30 секунди.

Заједничка подешавања рутера примењују слојеве из поглавља 4.3.1. CORS допушта локална порекла са креденцијалима (*cors_layer*), рок обраде је 10 секунди (*timeout_layer*), а обједињавање (*finalize_router*) доноси `/api`, `/docs` и границу тела захтева.

6.2 Сервис за кориснике

Сервис за кориснике је власник података о налозима и једини издавалац аутентификационих токена (*access* и *refresh*). Омогућава:

- регистрацију,
- пријаву,
- обнављање и повлачење токена,
- управљање профилем,
- измену лозинке,
- деактивацију налога и
- административне операције над улогама и блокирањем налога.

Јединственост корисничких имена, имејл адреса и токена обезбеђује се јединственим ограничењима на нивоу шеме, а корисник је обавештен одговарајућом поруком о грешци.

Лозинке се чувају хеширане алгоритмом `Argon2` (листинг 7, фајл *user-service/src/shared/password.rs*).

```
pub fn hash_password(password: &str) -> Result<String, ValidationError> {
    let salt = SaltString::generate(&mut OsRng);
    let hash = Argon2::default()
        .hash_password(password.as_bytes(), &salt)
        .map_err(|_| ValidationError::new(ERROR_MESSAGE))?;
    Ok(hash.to_string())
}
```

Листинг 7: Хеширање лозинке алгоритмом `Argon2` (*user-service/src/shared/password.rs*)

Токени се потписују претходно генерисаним паром RSA кључева смештеним у фолдер `playlog-backend/keys`, према поступку из поглавља 2.3. Приватни кључ поседује искључиво овај сервис, док јавни кључ користе сви сервиси за проверу пристиглог токена преко функције из сандука `jwt-common`.

`Access` токен носи идентификатор, улогу и основне податке корисника и важи подразумевано 300 секунди, док `refresh` токен носи само идентификатор корисника и важи подразумевано 14 дана, према подели из поглавља 2.3. Скуп информација се обликује конструкторима `Claims::for_access_token` и `Claims::for_refresh_token`, а потписивање је обједињено у функцији `create_tokens` (листинг 8).

```
pub fn create_tokens(access_token_validity: Duration, refresh_token_validity: Duration,
    jwt_private_key: &[u8], user_id: Uuid, role: Role, username: String, email: String) ->
    Result<(Tokens, DateTime<Utc>)> {
    let now = Utc::now();

    let expiration_date = now + access_token_validity;
    let claims = Claims::for_access_token(
        user_id.to_string(),
        expiration_date.timestamp() as usize,
        now.timestamp() as usize,
        role,
        username,
        email,
    );
    let access_token = create_token(claims, jwt_private_key)?;

    // refresh токен се генерише аналогно

    Ok((Tokens(access_token, refresh_token), expiration_date))
}
```

Листинг 8: Исечак функције за генерисање пара токена алгоритмом RS256 (скраћено из `user-service/src/auth/token.rs`)

Провера пристиглог токена налази се у сандуку `jwt-common` (листинг 9).

```
pub fn decode_token(token: &str, public_key: &[u8]) -> Result<Claims> {
    let decoding_key = DecodingKey::from_rsa_pem(public_key)
        .map_err(|err| JwtError::InvalidDecodingKey(err.to_string()))?;
    let mut validation = Validation::new(Algorithm::RS256);
    validation.iss = Some(HashSet::from([String::from(ISSUER)]));
    let token_data = decode::<Claims>(token, &decoding_key, &validation)
        .map_err(|err| JwtError::InvalidToken(err.to_string()))?;
    Ok(token_data.claims)
}
```

Листинг 9: Функција за декодирање JWT токена (`jwt-common/src/token.rs`)

Животним циклусом токена управља структура `AuthService` (`user-service/src/auth/service.rs`). При пријави се налог проналази по корисничком имену или имејл адреси, при чему се блокирани налог одбија, а деактивирани третира као непостојећи. Након тога се лозинка проверава према сачуваном хешу. Успешном пријавом издаје се нови пар токена, а `refresh` токен се уписује у базу са роком важења.

`Access` токен се враћа у телу одговора, док се `refresh` токен смешта у `HttpOnly` колачић (`user-service/src/shared/cookie.rs`), па није доступан скриптама на клијенту. Обнављање `access` токена спроводи ротацију описану у поглављу 2.3 (листинг 10): затечени токен се повлачи, а нови пар се издаје при чему се у базу уписује нови `refresh` токен. Одјавом се `refresh` токен повлачи у бази, а колачић се поништава.

```
pub async fn refresh_tokens(&self, config: &AppConfig, token: &str) -> Result<Tokens> {
    if !self.repository.is_token_valid(token).await? {
        return Err(AuthError::TokenError(String::from("Invalid token!")));
    }
    let claims = decode_token(token, &config.jwt_public_key)
        .and_then(RefreshTokenClaims::try_from)
        .map_err(|err| AuthError::TokenError(err.to_string()))?;
    self.revoke_token(token).await?;
    let user = self
        .repository
        .find_by_id(claims.user_id)
        .await?
        .ok_or(AuthError::UserNotFound)?;
    let tokens = self.generate_tokens(config, user).await?;
    Ok(tokens)
}
```

Листинг 10: Ротација *refresh* токена (user-service/src/auth/service.rs)

Истекли токени се периодично бришу позадинским задатком (user-service/src/task.rs), који се покреће при покретању сервиса и извршава једном дневно.

При покретању сервиса иницијализује се предефинисани администратор са креденцијалима уписаним у .env датотеци (user-service/src/main.rs, AuthService::ensure_admin). Налог се креира само када нема администратора у систему (при првом покретању), уз исте провере јединствености као при регистрацији, а рад сервиса се спречава све док се не креира налог предефинисаног администратора.

auth/ руте су јавне, при чему пријава и регистрација не траже токен, а одјава и обнављање имају колачић за *refresh* токен. Преглед туђег профила је јаван, личне операције стоје иза require_user, а претрага, промене улога и блокирање иза require_admin.

6.3 Сервис за каталог

Сервис за каталог представља централни извор података о играма. Чува основне податке о играма (назив, опис, датум изласка, веб сајт) и пратеће ентитете:

- програмере,
- издаваче,
- жанрове,
- платформе и
- ознаке,

при чему свака врста има засебну табелу. Игре пролазе кроз животни циклус од нацрта до објаве, при чему је нацрт видљив искључиво администраторима. Брисање је могуће само док је игра нацрт.

Интегритет на нивоу шеме базе података обезбеђују јединствени страни кључеви са каскадним брисањем у везама игара са пратећим ентитетима. Примењени су индекси из поглавља 4.2.1 (листинг 11): GIN триграмски индекси за текстуалну претрагу назива и индекси над страним кључевима за спајања са пратећим ентитетима.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX idx_games_name ON games USING gin (name gin_trgm_ops);
CREATE INDEX idx_developers_name ON developers USING gin (name gin_trgm_ops);
CREATE INDEX idx_game_developers_developer_id ON game_developers (developer_id);
CREATE INDEX idx_game_tags_tag_id ON game_tags (tag_id);
```

Листинг 11: Исечак индекса сервиса за каталог (catalogue-service/migrations/20260218212643_init.up.sql)

Trait `GameRepository` (листинг 12) прописује операције над играма, док пратећи ентитети деле *trait*-ове `GameEntityRepository` (операције `SELECT`, `INSERT`, `UPDATE`) и `DeletableGameEntityRepository` (операција `DELETE`). Ентитети програмера и издавача не могу да се бришу.

```
[async_trait]
pub trait GameRepository: Send + Sync {
    async fn filter(
        &self,
        include_drafts: bool,
        params: GameFilterQuery,
    ) -> Result<Vec<GameSimple>>;
    // ...
    async fn get_details(&self, id: i32, include_draft: bool) -> Result<Option<GameDetails>>;
    async fn create(&self, data: CreateGameRequest) -> Result<Game>;
    async fn update(&self, id: i32, data: UpdateGameRequest) -> Result<GameDetails>;
    async fn publish(&self, id: i32, version: i64) -> Result<Game>;
    async fn delete(&self, id: i32, version: i64) -> Result<()>;
}
```

Листинг 12: Исечак `GameRepository trait`-а (скраћено из `catalogue-service/src/games/repository.rs`)

Структура `GameService` чува `Box<dyn GameRepository>` и репозиторијуме пратећих ентитета као `Box<dyn GameEntityRepository>`, али пословну логику спроводи само за игре: репозиторијуми ентитета служе искључиво за проверу постојања ентитета при додавању или измене игре (`catalogue-service/src/app.rs`).

Пратећи ентитети игре деле заједнички модел (`GameEntity`) и имплементацију репозиторијума (`PostgresGameEntityRepository`). Не захтевају посебан сервисни слој, па њихови *handler*-и користе репозиторијуме директно.

Упит за претрагу са филтрирањем игара методе `GameRepository::filter` се склапа помоћу `QueryBuilder`: полази се од `DISTINCT` уз `LEFT JOIN` ка везама ознака, жанрова и платформи, затим се додају опциони `IN` филтери и `LIKE` услов са пречишћеним обрасцем. Сортирање је задато набројивим типом, а странице обухватају до 10 записа. Детаљи игре се добијају методом `GameRepository::get_details`, једним упитом за игру и са пет упита за пратеће ентитете.

Методе `create` и `update` извршавају се трансакционо, како би ред игре и везе ка пратећим ентитетима заједно успели или заједно отказали: ред игре се упише или измени, а затим помоћне методе `set_*` обнављају везе брисањем по игри и поновним уписом по идентификатору. При измени се дирају само прослеђене `Option<Vec<i32>>` везе. Структура `GameService` пре уписа проверава идентификаторе: изостављени пролазе, празан скуп се одбија, а сваки наведени мора постојати.

Методе `publish`, `update` и `delete` користе верзију записа за оптимистично закључавање [17]. Објављивање (листинг 13) у једном `WHERE` спаја проверу верзије и заштиту нацрта, уз увећање верзије. Ако ниједан ред није измењен, накнадни упит разликује узроке: непостојећа игра је `NotFound`, објављена је `AlreadyPublished`, а измењена у међувремену је `Conflict`. Конфликти се пресликавају у одговор 409. Исти образац важи за измену игре и `update_name` пратећих ентитета, где се назив табеле добија из `table_name` уз `AssertSqlSafe`.

```

let game = query_as!(
    Game,
    r#"
        UPDATE games
        SET draft = false, version = version + 1
        WHERE id = $1 AND version = $2 AND draft = true
        RETURNING id, name, description, released, website, draft, version
    "#,
    id,
    version
)
.fetch_optional(&self.pool)
.await?;
match game {
    Some(game) => Ok(game),
    None => {
        // SELECT draft ...: нема реда = NotFound,
        // објављена = AlreadyPublished, нацрт = Conflict
    }
}

```

Листинг 13: Објављивање игре уз проверу верзије (скраћено из catalogue-service/src/games/repository.rs)

Руте су подељене према *middleware*-има који их штите (листинг 14): јавне GET руте стоје иза *auth_optional*, а уписи и преглед необјављених игара иза *auth* и *require_admin*. Из *AuthClaims* се преузима улога, па администратор у претрази и детаљима види и нацрте, а остали само објављене игре.

```

pub fn router(state: Arc<AppState>) -> OpenApiRouter<Arc<AppState>> {
    let jwt_config = JwtConfig::new(state.config.jwt_public_key.clone());
    let public_routes = OpenApiRouter::new()
        .route("/filter", get(filter))
        .route("/{id}/details", get(get_details))
        // ... споредне GET руте
        .route_layer(from_fn_with_state(jwt_config.clone(), auth_optional));
    let admin_routes = OpenApiRouter::new()
        .route("/", post(create))
        .route("/unpublished", get(get_unpublished))
        .route("/{id}", put(update))
        .route("/{id}", delete(delete_game))
        .route("/{id}/publish", put(publish))
        .route_layer(from_fn(require_admin))
        .route_layer(from_fn_with_state(jwt_config, auth));
    OpenApiRouter::new().merge(public_routes).merge(admin_routes)
}

```

Листинг 14: Раздвајање јавних и администраторских рута (скраћено из catalogue-service/src/games/handler.rs)

6.4 Сервис за библиотеку

Библиотека бележи везу између корисника и игре у табели *user_games*, уз један од пет статуса из поглавља 3.2.3. Јединствене везе на нивоу шеме обезбеђује сложени примарни кључ (корисник, игра).

Упис најпре проверава постојање игре преко *CatalogueClient*: непостојање у каталогу постаје *InvalidGameId* (404), а сваки други исход *CatalogueServiceError* (502). Додавање и измена статуса изводе се једним уписом (листинг 15): при конфликту на сложеном кључу освежавају се само статус и време измене, а ред се враћа.

```

let game = query_as!(
    UserGame,
    r#"
    INSERT INTO user_games (user_id, game_id, status)
    VALUES ($1, $2, $3)
    ON CONFLICT (user_id, game_id) DO UPDATE
    SET status = EXCLUDED.status, last_updated = now()
    RETURNING user_id, game_id, status AS "status: GameLibraryStatus", added_at, last_updated
    "#,
    user_id,
    game_id,
    status as _
)
.fetch_one(&self.pool)
.await?;
Ok(game)

```

Листинг 15: Упис или измена статуса једним упитом (library-service/src/repository.rs)

Читање се изводи преко COUNT, па SELECT са опционим филтером статуса, од најскорије измењених, уз ограничену страницу и границу, у облику PagedResponse.

Брисање без погођеног реда враћа NotFound, а нарушавања јединствености и страног кључа мапирају се у 409.

Идентификатор корисника за упис преузима се искључиво из Extension<AuthClaims>, никада из путање или тела захтева, па корисник има право да мења само сопствене редове. Руте за операције над библиотеком су заштићене помоћу require_user, а читање је јавно са идентификатором из путање.

6.5 Сервис за кориснички садржај

Садржај чине три колекције докумената са денормализованим корисничким именима:

- оцене (четири вредности из 3.2.4) са опционим текстом рецензије,
- коментари са циљем (игра или оцена) и
- пријаве са циљем (коментар или оцена), подносиоцем, разлогом и животним циклусом.

Једна активна оцена по кориснику и игри, као и једна пријава на чекању, обезбеђују се делимично јединственим индексима на нивоу шеме. Читања убрзавају сложени индекси према игри, оцени и циљу. Упис оцене најпре проверава постојање игре у каталогу (404/502). Затечена оцена се преписује условно верзијом, а нова убацује са почетном верзијом (листинг 16). Неуспела замена због неслагања верзије пресликава се у Conflict.

```

match review.id {
    Some(id) => {
        let current_version = review.version;
        review.version += 1;
        let filter = doc! { "_id": id, "version": current_version };
        let result = self.reviews.replace_one(filter, review.clone()).await?;
        if result.matched_count == 0 {
            return Err(ReviewError::Conflict(id));
        }
        Ok(review)
    }
    None => {
        // ... insert_one са преузимањем додељеног идентификатора ...
    }
}

```

Листинг 16: Упис оцене уз проверу верзије (скраћено из review-service/src/review/repository.rs)

Измена и брисање учитавају документ и пореде аутора са идентитетом из токена. Брисање је логичко: поставља `deleted` уз увећање верзије, а обрисано се чита као непостојеће.

Подношење пријаве одбија дуплу пријаву на чекању поднету од стране истог корисника за исти садржај. Разрешење (листинг 17) одбија разрешавање сопствене пријаве и допушта само прелазе са чекања: уважавање логички брише циљани документ (оцену или коментар) кроз његов репозиторијум, а ред пријаве се пребацује уз проверу верзије, па двострука модерација завршава конфликтом.

```
let report = self.report_repository.find_by_id(id).await?
    .ok_or(ReportError::NotFound)?;
if report.reporter_id == resolver_id {
    return Err(ReportError::CantResolveOwnReport);
}
if report.status != ReportStatus::Pending {
    return Err(ReportError::IllegalStatus(report.status.as_db_value()));
}
if let ReportStatus::Resolved = status {
    // ... логичко брисање циља кроз репозиторијум рецензија или коментара ...
}
self.report_repository.update_report_status(id, status, version).await
```

Листинг 17: Разрешавање пријаве (скраћено из `review-service/src/report/service.rs`)

Збирови се израчунавају агрегацијама (`$group`, `$avg`, `$sort`): статистика по оценама, најбоље оцењене и највише оцењиване игре, уз број коментара по рецензији. Читања су јавна, док уписи рецензија, коментара и пријава стоје иза `require_user`, а листа чекања и разрешавање иза `require_moderator`.

6.6 Сервис за мултимедију

Сервис за мултимедију чува слике и видео записе игара: бајтове у објектном складишту, а метаподатке у бази докумената. Један медијски документ по игри обезбеђује се јединственим индексом на нивоу шеме. Документ носи верзионисане метаподатке (насловну слику, снимке екрана и трејлер као записе са кључем објекта, MIME типом и величином), док су бајтови фајлова у објектном складишту. Три именована поља отпремања (`cover`, `screenshot`, `trailer`) мапирају се у набројив тип `FieldName`.

Најпре се проверава постојање игре (404/502). Отпремање затим ограничава садржај: највише 22 датотеке (насловна слика, трејлер и максимално 20 снимака екрана). Сlike су ограничене на 10 MB, а видео на 500 MB, уз проверу MIME типа по пољу. Отпремање се одвија под привременим кључевима са ознаком покушаја, а први упис настаје са почетном верзијом. Истовремене прве отпреме сударају се на јединственом индексу: пролази само једна.

При измени затеченог документа верзија се најпре проверава у меморији, пре постављања бајтова у објектно складиште. Затим следе постављање и спајање новог садржаја са затеченим документом уз увећање верзије. Ознака покушаја раздваја истовремене отпреме под различите кључеве, па кроз верзионисани упис пролази само једна. Неуспешан верзионисани упис (листинг 18) враћа `Conflict` и брише управо постављене кључеве, а замењени (стари) кључеви бришу се тек по успеху.

```

self.validate_upload_limits(&files)?;
let existing = self.repository.find_by_game_id(game_id).await?
    .unwrap_or_else(|| GameMedia::new_for_game(game_id));
if existing.version != version {
    return Err(MediaError::Conflict(game_id));
}
let (new_cover, new_screenshots, new_trailer) =
    self.process_and_upload_files(game_id, files).await?;
// ... спајање новог са затеченим уз version + 1 ...
if let Err(error) = self.repository.upsert(media, version).await {
    self.storage.delete_keys(&uploaded_keys).await;
    return Err(error);
}
self.storage.delete_keys(&replaced_keys).await;

```

Листинг 18: Отпремање са провером верзије и враћањем постављеног (скраћено из `multimedia-service/src/service.rs`)

Кључна одлика је да сервис при читању никада не прослеђује бајтове кроз себе, већ за сваки тражени фајл генерише URL са роком важења од једног сата, према приступу из поглавља 4.3.6 (листинг 19):

```

async fn presign(&self, object_key: &str) -> Result<String> {
    self.client
        .get_presigned_object_url(&self.bucket, object_key, Method::GET)
        .expiry_seconds(60 * 60)
        .send()
        .await
        .map(|response| response.url)
        .map_err(|e| MediaError::StorageError(e.to_string()))
}

```

Листинг 19: Стварање унапред потписаног URL-а ка објектном складишту (`multimedia-service/src/storage.rs`)

Групе насловне слике разрешавају се једним `$in` упитом и враћају се као мапа идентификатор-URL, где недостајући запис добија вредност `null` (до 100 идентификатора, празан улаз даје празан излаз).

Брисање проверава верзију у меморији, затим верзионисано брише документ (непостојећи је `NotFound`, затечени `Conflict`), па брише кључеве из складишта.

Рок обраде (600 секунди) и граница тела захтева (512 MB) прилагођени су величини видео записа. Грешке валидације су 400. Непостојање, конфликт и каталог носе 404, 409 и 502, а откази складишта и базе прикривени су 500 уз евиденцију. Читања су јавна, а отпремање и брисање чува `require_admin`.

6.7 API gateway сервис

API gateway је по обрасцу из поглавља 2.2 једина улазна тачка платформе и једини сервис без складишта. За остале сервисе иницијализује посебно стање `ServiceAppState`: адресу сервиса, дељени `ProxyClient` и `JwtConfig`. Дељени клијент носи рок обраде и границу тела захтева прилагођене отпремањима видеа која пролазе кроз `proxy` ка сервису за мултимедију.

Рутер за сваки сервис понавља путање тог сервиса, а све показују на универзални `proxy_handler` (листинг 20). Јавне руте немају заштиту, а заштићене стоје иза `auth` и одговарајуће провере улоге. Одбијени захтев никада не стиже до сервиса, али се заглавље `Authorization` прослеђује даље, па сервис поновно проверава потпис, по обрасцу `zero trust` [12].

```
#[debug_handler]
pub async fn proxy_handler(
    State(state): State<Arc<ServiceAppState>>,
    OriginalUri(original_uri): OriginalUri,
    request: Request,
) -> Result<Response> {
    let (parts, body) = request.into_parts();
    let path = original_uri.path();
    let path = if let Some(query) = parts.uri.query() {
        format!("{:?}{}", path, query)
    } else {
        path.to_string()
    };
    state
        .proxy_client
        .forward_request(&state.service_url, &path, parts.method, parts.headers, body)
        .await
}
```

Листинг 20: универзални *proxy handler* за *endpoint*-ове свих сервиса (`api-gateway/src/proxy/handler.rs`)

Прослеђивање (листинг 21) саставља адресу, одбацује заглавља везе и регенерисана заглавља (Host, Content-Length). Преостала заглавља чува, укључујући и Authorization. Тело се стримује без баферовања, па видео пролази без оптерећења меморије. Одговор се враћа са статусом, заглављима и бајтовима.

```
pub async fn forward_request(
    &self,
    service_url: &str,
    path: &str,
    method: Method,
    headers: HeaderMap,
    body: Body,
) -> Result<Response> {
    let base = service_url.trim_end_matches('/');
    let url = format!("{}/{}", base, path);
    let mut request = self.client.request(method.clone(), &url);
    for (header_name, header_value) in headers
        .iter()
        .filter(|(header_name, _)| should_forward_header(header_name.as_str()))
    {
        request = request.header(header_name, header_value);
    }
    request = request.body(request::Body::wrap_stream(body.into_data_stream()));
    let response = request.send().await.map_err(|e| {
        GatewayError::ServiceUnavailable(format!("Failed to reach service: {}", e))
    })?;
    // Конвертовање request::Response у axum::Response
    Self::convert_response(response).await
}
```

Листинг 21: Прослеђивање захтева ка сервису (`api-gateway/src/proxy/client.rs`)

Грешка `GatewayError` има сопствени облик `{"error"}`: недоступан сервис је 503, а нечитљив одговор 502. Алијас типа `Result` прати образац осталих целина.

При покретању *gateway* преузима *OpenAPI* спецификацију сваког сервиса и спаја их у јединствени приказ (листинг 22), а неуспело преузимање само се евидентира.

```
pub async fn load_service_docs(config: &Config, client: &Client) -> OpenApiDocument {
    let mut merged = ApiDoc::openapi();
    let service_urls: Vec<&str> = vec![
        // ... адресе свих сервиса ...
    ];
    for base_url in service_urls {
        match fetch_service_doc(client, base_url).await {
            Ok(spec) => merged.merge(spec),
            Err(err) => error!("Failed to load OpenAPI spec from {}: {err}", base_url),
        }
    }
    merged
}
```

Листинг 22: Спајање спецификација сервиса (скраћено из `api-gateway/src/docs.rs`)

6.8 Документација API-ја

Принцип генерисања документације је описан у поглављу 4.3.7. Сваки сервис пријављује своје *handler*-е у декларацију `ApiDoc` (листинг 23). Омотач `OpenApiRouter` прикупља спецификацију уз регистрацију рута, а додаток `SecurityAddon` пријављује *bearer* шему за заштићене руте. Приказани *Swagger* UI омогућава испробавање рути сервиса пре израде клијента.

```
#[utoipa::path(
    post,
    path = "/api/auth/register",
    request_body = RegisterRequest,
    summary = "Register new user",
    responses(
        (status = 201, description = "Successful register", body = RegisterResponse),
        (status = 400, description = "Request has missing values, or the values are
invalid"),
        (status = 409, description = "Email or username is already taken"),
    ),
    tag = "auth",
)]
#[debug_handler]
pub async fn register(
    State(state): State<Arc<AppState>>,
    Json(request): Json<RegisterRequest>,
) -> ApiResult<impl IntoResponse> {
    request.validate().map_err(ApiError::from)?;
    let result = state.auth_service.register(request).await?;
    Ok((StatusCode::CREATED, Json(result)).into_response())
}
```

Листинг 23: Декларативни *OpenAPI* макро *handler*-а за регистрацију (исечак из `user-service/src/auth/handler.rs`)

6.9 Клијентска апликација

Клијентска апликација обрађена је кроз радни оквир *Angular* у поглављу 6.9.1 и веб сервер *nginx* у поглављу 6.9.2. У развоју *nginx* се не користи, већ *Angular development* сервер директно упућује захтеве ка *API gateway*-у.

6.9.1 Angular

Клијент је изграђен од самосталних (енгл. *standalone*) компоненти, са засебном компонентом по страници. *Access* токен се складишти у `localStorage` (кључ `playlogAccessToken`), а *refresh* токен искључиво у `_HttpOnly_` колачићу уз `withCredentials`. Корисник се одређује декодирањем токена, а без токена је

гост. Пријава на систем складишти токене, одјава га уклања уз повлачење *refresh* токена на серверу. При покретању апликације и освежавању странице на претраживачу затечени истекли токен се освежава.

Пресретац `AuthInterceptor` на све руте осим четири `auth/` руте додаје `Bearer` заглавље. При истеку токена најпре освежава одвојеним `HTTP` клијентом (који заобилази `AuthInterceptor`, како не би дошло до рекурзије), уз обједињавање истовремених освежавања, па понавља захтев (листинг 24). Неуспешно освежавање одјављује корисника, враћа га на почетну страницу и приказује обавештење. Истеклоост се проверава пре слања захтева, па се одговор 401 никада не понавља.

```
if (this.sessionService.isTokenExpired()) {
  return this.sessionService.refreshToken().pipe(
    switchMap((newToken) => {
      const cloned = req.clone({
        setHeaders: { Authorization: `Bearer ${newToken}` },
      });
      return next.handle(cloned);
    }),
    catchError((err) => throwError(() => err))
  );
}
```

Листинг 24: Освежавање и понављање захтева (скраћено из `app/core/interceptors/auth.interceptor.ts`)

Чувар `authGuard` проверава улогу корисника према тренутној рути и враћа на почетну страну при било каквом неслагању (листинг 25).

```
export const authGuard: CanActivateFn = (route: ActivatedRouteSnapshot) => {
  const sessionService = inject(SessionService);
  const router = inject(Router);
  const user = sessionService.user();
  const roles = route.data['roles'] as Role[];
  if (!roles || roles.length === 0) {
    return true;
  }
  if (!roles.includes(user.role)) {
    router.navigate(['home']);
    return false;
  }
  return true;
};
```

Листинг 25: Заштита руте према улози (скраћено из `app/core/guards/auth.guard.ts`)

6.9.2 *nginx*

Датотека `nginx.conf` примењује улоге описане у поглављу 4.5: статичке датотеке сервера са диска уз повратак на `index.html` за клијентске руте. Почетна страница се не кешира зато што је *Angular* једностранична (енгл. *single page*) апликација, а хеширане датотеке су непроменљиве годину дана.

`/api/` прослеђује ка адреси из окружења уз заглавља прослеђивања, чиме релативни `/api` стиже до *API gateway*-а. Такође прослеђује захтеве упућене објектном складишту (које није изложено изван *Docker* мреже у продукцији), па прегледач слике са `img src` адреса добавља директно из складишта.

`nginx.conf` садржи и ограничење величине тела захтева од 500 MB, према највећем дозвољеном отпремању, уз прикривање верзије сервера.

6.10 Покретање платформе

Docker слике сервиса се граде вишестепено (листинг 26), према поступку из поглавља 4.6: извршавање у *alpine* слици као непривилеговани корисник, са бинарним фајлом и јавним кључем. Приватни кључ уграђује само слика сервиса за кориснике. Сваки сервис носи сопствену *Dockerfile* датотеку, па се слике граде и објављују независно. Провера здравља позива одговарајућу руту сервера и пријављује исправност контејнера у току рада.

```
ARG RUST_VERSION=1.95
ARG APP_NAME=user-service
FROM rust:${RUST_VERSION}-alpine AS build
ARG APP_NAME
WORKDIR /app
RUN apk add --no-cache clang lld musl-dev git curl
RUN --mount=type=bind,source=services,target=services \
  --mount=type=bind,source=shared,target=shared \
  SQLX_OFFLINE=true cargo build --locked --release -p $APP_NAME \
  && cp ./target/release/$APP_NAME /bin/server
FROM alpine:3.24 AS final
RUN apk add --no-cache wget
RUN adduser --disabled-password --uid "10001" appuser
USER appuser
COPY --from=build /bin/server /bin/
COPY --chown=appuser:appuser keys/private.pem /keys/private.pem
COPY --chown=appuser:appuser keys/public.pem /keys/public.pem
HEALTHCHECK --interval=10s --timeout=5s --retries=10 \
  CMD wget -q -O /dev/null http://127.0.0.1:3005/api/user-service-health || exit 1
CMD ["/bin/server"]
```

Листинг 26: Вишестепена слика серверског сервиса (скраћено из *user-service/Dockerfile*)

Клијентска слика гради се алатом *Bun* [42], а сервира је непривилеговани *nginx* на порту 8080, са адресом *API gateway*-а учитаном из окружења (листинг 27).

```
FROM oven/bun:1-alpine AS builder
WORKDIR /app
COPY package.json bun.lock ./
RUN bun install --frozen-lockfile
COPY angular.json tsconfig*.json ./
COPY src ./src
COPY public ./public
RUN bun run build
FROM nginxinc/nginx-unprivileged:1.31-alpine AS runner
ENV NGINX_ENVSUBST_FILTER=BACKEND_URL
COPY --chown=nginx:nginx nginx.conf /etc/nginx/templates/nginx.conf.template
COPY --chown=nginx:nginx --from=builder /app/dist/*/browser /usr/share/nginx/html
USER nginx
EXPOSE 8080
CMD ["nginx", "-g", "daemon off;"]
```

Листинг 27: Слика клијентске апликације (скраћено из *playlog-frontend/Dockerfile*)

Compose датотеке описују апликативне и инфраструктурне контејнере заједно: сваки сервис прилаже сопствену датотеку, повезану заједничком мрежом *playlog_network*, са именованим волуменима за податке, при чему провере складишта (*pg_isready*, *mongosh* *ping*, *MinIO* провера) задржавају покретање зависних сервиса.

Локално покретање користи *.env*, а покретање преко *Docker Compose*-а *.env.production*, са *DNS* именима контејнера уместо локалних адреса. *Makefile* нуди основне циљеве *start*, *stop* и *logs*, у заједничкој и

појединачној варијанти (start-user-service и слично). Изградња (build, rebuild) је паралелна, а сервисни Makefile-ови додају start-db за саму базу и start-dev за зависности са чекањем на здравље. Пар кључева је претходно генерисан у фолдеру playlog-backend/keys и уграђује се при изградњи. Изложени портови су дати у табели 4: у продукцији само 8080, у развоју и портови складишта.

Ван контејнера, развојни *Angular* клијент ради на порту :4200, а сервиси покренути командом cargo на портовима :3000-:3005.

Табела 4: Изложени портови по окружењу

Контејнер	Порт контејнера	Мапирани порт (продукција)	Мапирани порт (развој)
API gateway сервис	3000	–	–
Сервис за каталог	3001	–	–
Сервис за библиотеку	3002	–	–
Сервис за мултимедију	3003	–	–
Сервис за кориснички садржај	3004	–	–
Сервис за кориснике	3005	–	–
Клијент	8080	8080	–
PostgreSQL (3 базе)	5432	–	5433-5435
MongoDB (2 базе)	27017	–	27018-27019
MinIO (конзола 9001)	9000	–	9000, 9001

У овом поглављу је приказано коришћење платформе кроз клијентску апликацију. Прво је објашњен начин припреме демонстрационих података у потпоглављу 7.1. Затим следе кориснички токови у потпоглављу 7.2. На крају су истакнуте административне операције у потпоглављу 7.3. Сваки одељак полази од корисничких корака видљивих на снимцима екрана и укратко описује ток захтева од клијентске апликације преко API gateway-а до одговарајућих микросервиса и њихових база података.

7.1 Припрема демонстрационих података

Сви подаци о играма и мултимедијални садржај приказани у остатку поглавља преузети су са платформе RAWG преко њеног API-ја [7] и служе искључиво за демонстрацију. Сами кориснички налози, личне библиотеке корисника, коментари, оцене и пријаве непримереног садржаја су насумично генерисани и убачени у базе помоћу Python скрипти.

Након извршавања Python скрипти, базе су довољно напуњене подацима за реалистичан приказ пагинације и претраге. Обим припремљених података по сервису сажет је у табели 5. Број редова из асоцијативних табела у сервису за каталог није наведен, јер садрже само парове идентификатора.

Табела 5: Обим припремљених демонстрационих података

Сервис	Складиште	Садржај	Обим
Сервис за кориснике	PostgreSQL	налози	~500
Сервис за каталог	PostgreSQL	игре и пратећи ентитети	~200 и ~750
Сервис за библиотеку	PostgreSQL	везе корисник-игра	~55 000
Сервис за кориснички садржај	MongoDB	оцене, коментари и пријаве садржаја	~38 000, ~76 000 и ~20 000
Сервис за мултимедију	MongoDB + MinIO	метаподаци и објекти (~1,2 GB)	~200 и ~800

7.2 Кориснички токови

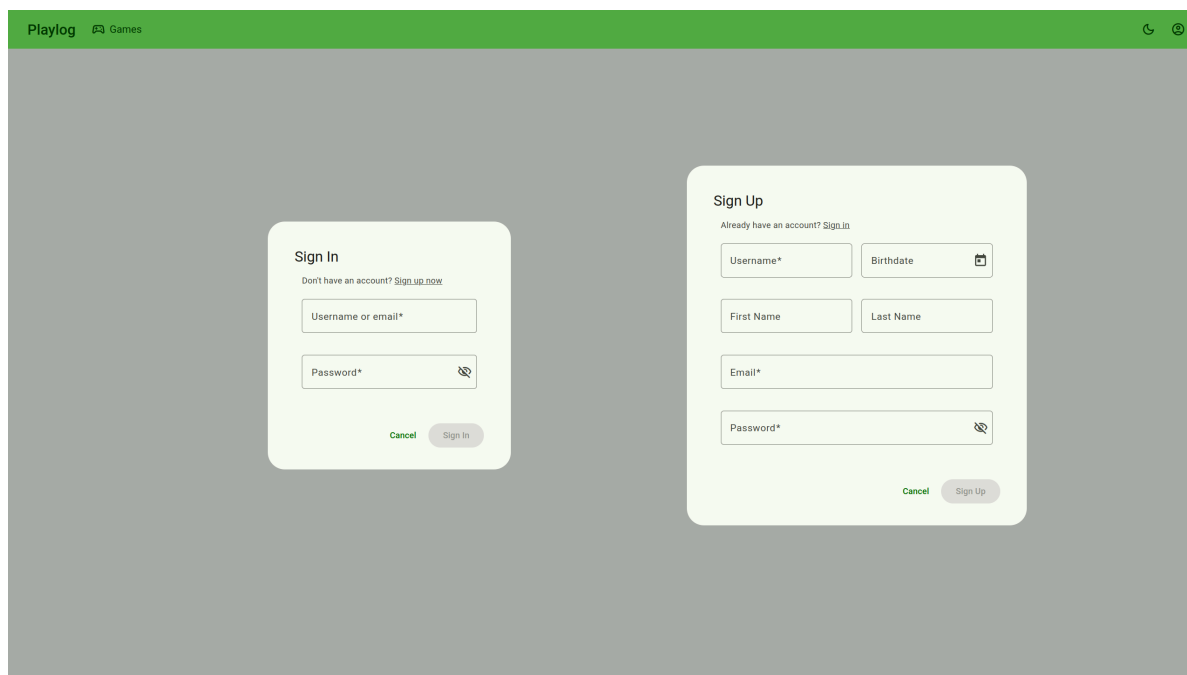
7.2.1 Почетна страница

При отварању платформе корисник се налази на почетној страници са прегледом најновијих, највише оцењиваних и најбоље оцењених игара. Приказане су и скорашње рецензије и коментари. Подаци се добављају из сервиса за кориснички садржај и сервиса за каталог. Пријављени корисници виде и своје игре из личне библиотеке у статусу „тренутно се игра“, добављене из сервиса за библиотеку.

7.2.2 Регистрација и пријава на платформу

Трака за навигацију је на свим страницама и прилагођава се статусу и улози: госту се нуди отварање дијалога за пријаву на платформу и регистрацију преко менија за налог, а пријављеном кориснику мени са личним налогом и библиотеком, уз администраторске и модераторске ставке према улози.

Гости платформе се региструју уносом минимално корисничког имена, имејл адресе и лозинке, док се регистровани корисници пријављују на платформу уносом корисничког имена или имејл адресе и лозинке. На слици 2 су приказана оба дијалога, уз напомену да се у једном тренутку може отворити само један (представљени су на истој слици само ради поређења).



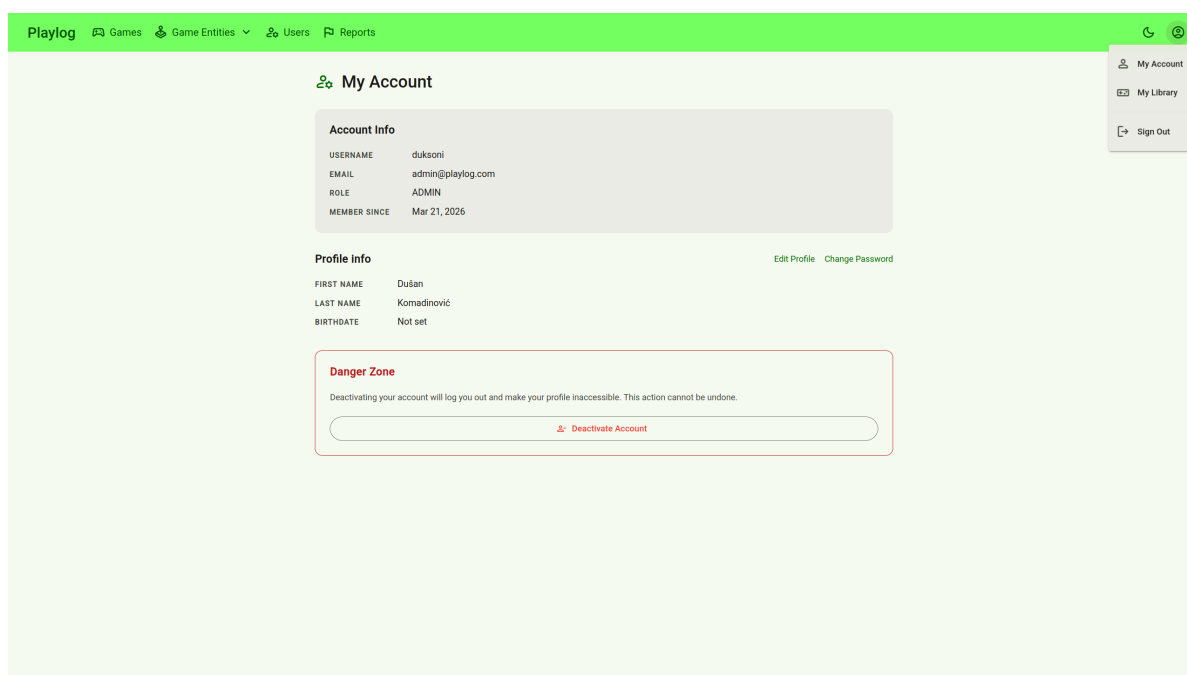
Слика 2: Дијалози за регистрацију и пријаву на платформу.

Успешном пријавом корисник добија пар токена описаних у поглављима 2.3 и 6.2. *Access* токен се чува у *localStorage*-у прегледача, док се *refresh* токен чува у *HttpOnly* колачићу. Одјавом се *refresh* токен повлачи на серверу, а оба трага сесије се уклањају на клијенту.

Регистрација, пријава, одјава и обнављање пара токена се врше позивима `POST /api/auth/register`, `POST /api/auth/login`, `POST /api/auth/logout` и `POST /api/auth/refresh`, које обрађује сервис за кориснике.

7.2.3 Управљање личним налогом

Странице личног налога, приказаној на слици 3, приступа се преко менија налога у траци за навигацију. Пријављени корисник прегледа податке о налогу, може да их мења и да деактивира налог.



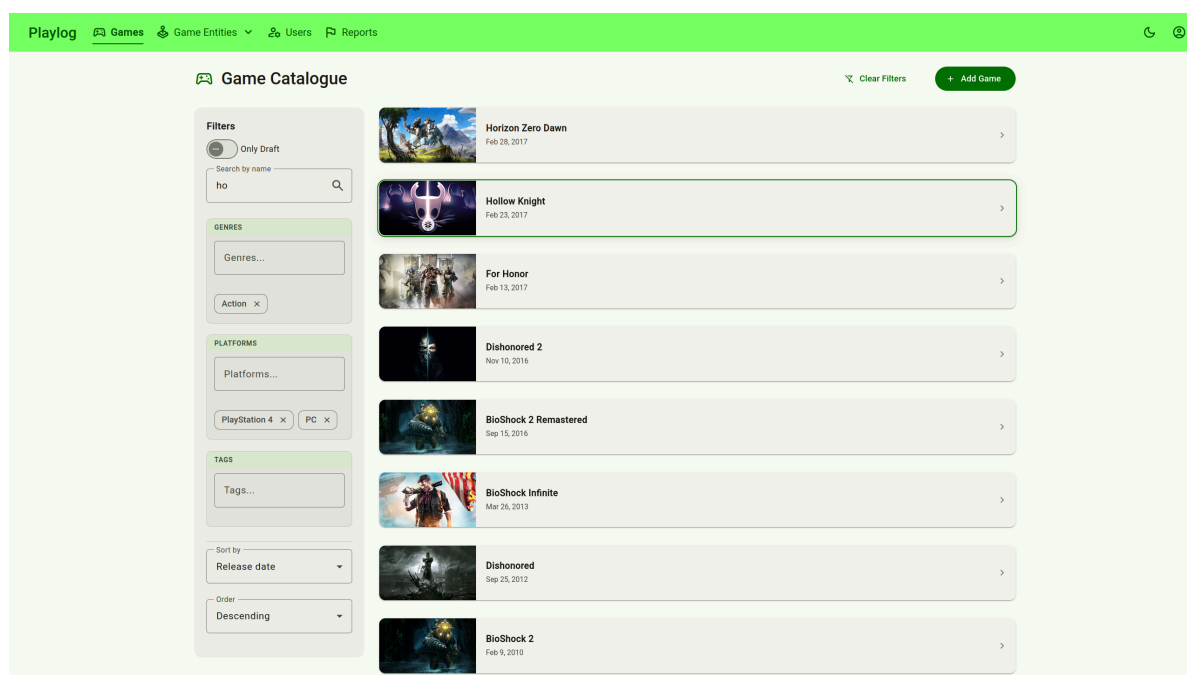
Слика 3: Страница личног налога.

Подаци се добављају позивом GET /api/users/:username, а измене позивима PUT /api/users/me, PUT /api/users/me/change-password и DELETE /api/users/me. Сервис за кориснике измене спроводи над базом, уз проверу верзије записа (конфликт је могућ једино кад администратор промени улогу корисника или га блокира истовремено док корисник покуша да измени податке на налогу).

7.2.4 Претрага каталога

Страницу каталога корисник отвара из траке за навигацију, на којој може да претражује игре по називу, уз филтрирање и сортирање. Филтрирање је омогућено по платформама, жанровима и ознакама. Како корисник помера страницу надолу, садржај се постепено учитава.

Садржај је доступан и без пријаве на платформу. На слици 4 приказана је страница каталога са примењеним филтерима и сортирањем, одакле се кликом на картицу игре отвара страница са детаљима.



Слика 4: Претрага и филтрирање каталога.

Претрага се извршава кроз сервис за каталог. Компонента GamesListPage прикупља параметре претраге и позива GET /api/games/filter. У репозиторијуму се упит динамички склапа према задатим филтерима и извршава над базом, уз пагинацију.

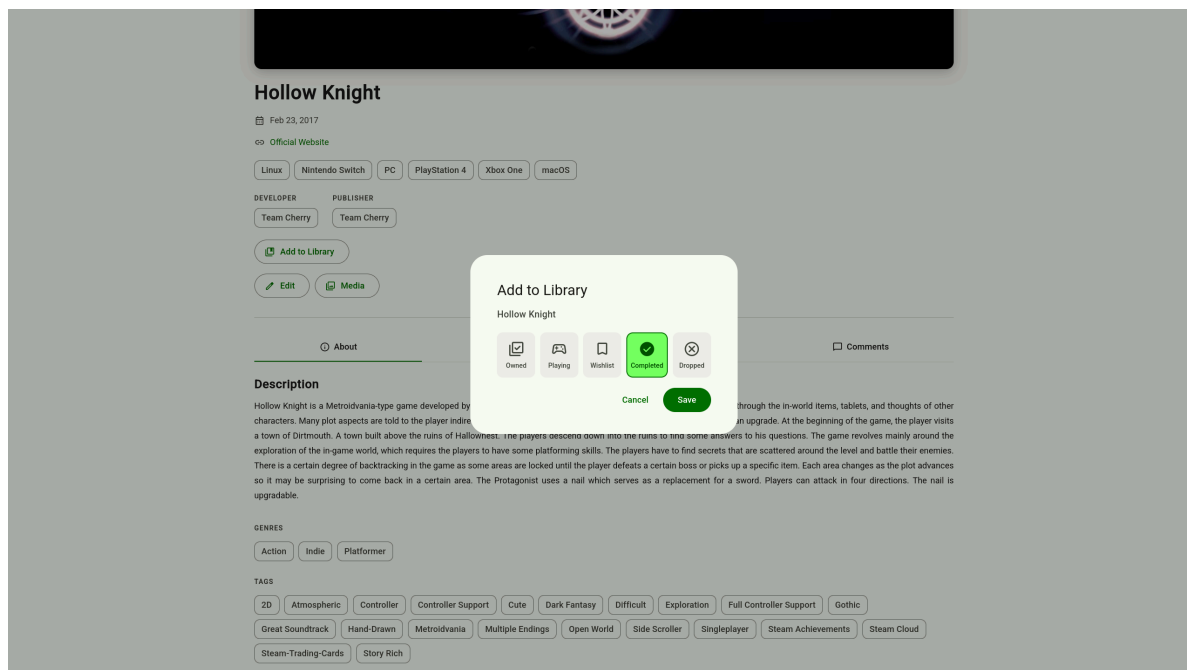
Насловне слике за добављене игре се накнадно довлаче групним позивом GET /api/media/games/covers, како је описано у поглављу 6.6.

7.2.5 Приказ детаља игре и управљање библиотеком

Страница са детаљима игре је главни пример обједињеног садржаја из више сервиса - компонента GameDetailPage паралелно позива:

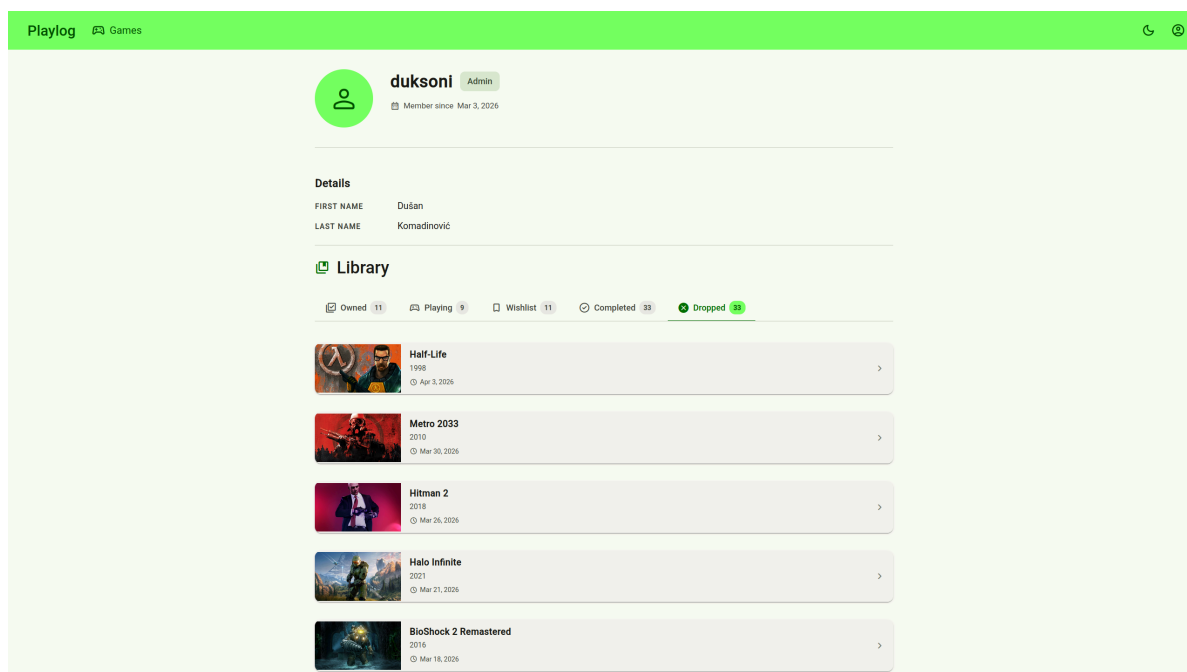
- сервис за каталог (GET /api/games/:id/details) да добави основне податке,
- сервис за мултимедију (GET /api/media/games/:gameId) да добави слике и видео,
- сервис за кориснички садржај да добави:
 - оцене (GET /api/reviews/game/:gameId),
 - број оцена по типовима (GET /api/reviews/game/{gameId}/stats) и
 - коментаре (GET /api/comments?targetType=GAME&targetId=:gameId),
- сервис за библиотеку да добави статус у библиотеци (GET /api/library/user/:userId).

На страници са детаљима игре, пријављени корисник може да дода игру у личну библиотеку у један од пет статуса из поглавља 3.2.3 кликом на одговарајуће дугме, при чему ће се отворити дијалог приказан на слици 5.



Слика 5: Страница детаља игре са отвореним дијалогом за додавање игре у личну библиотеку.

Странице личне библиотеке се приступа преко менија налога у траци за навигацију, на којој додата игра постаје видљива. Библиотека је такође видљива свим корисницима на страници јавног профила, приказаној на слици 6.



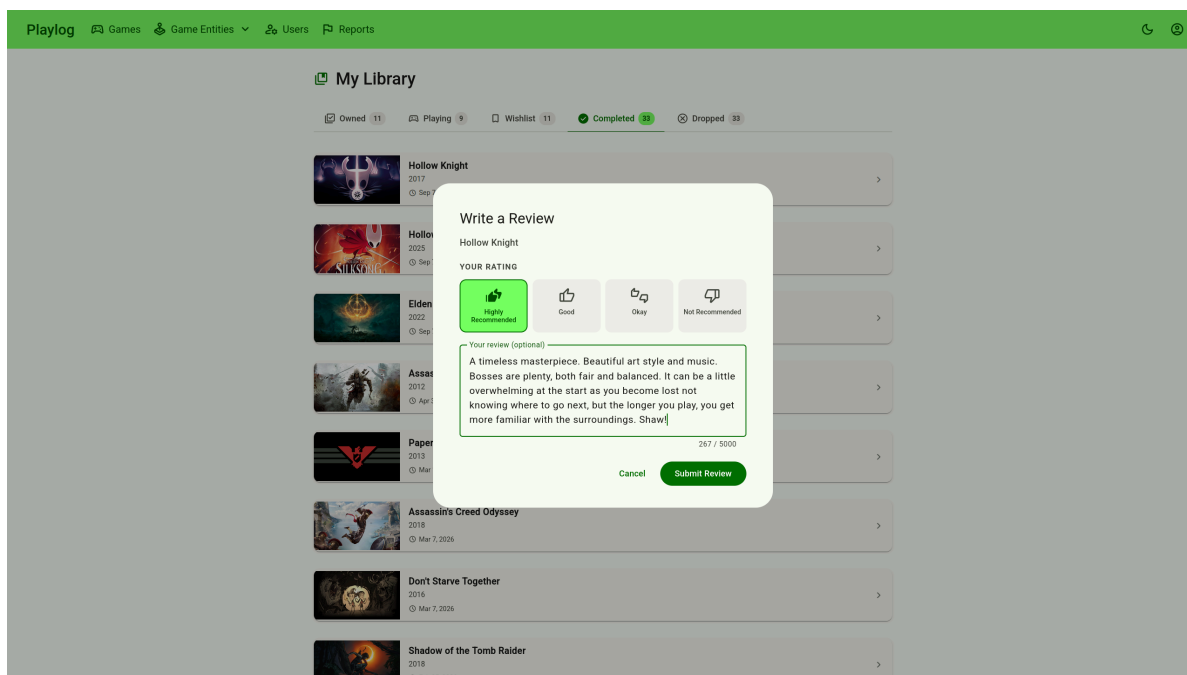
Слика 6: Библиотека на страници јавног профила корисника.

Упис у библиотеку се извршава позивом `POST /api/library`. Сервис за библиотеку пре уписа проверава постојање игре преко сервиса за каталог, како је описано у поглављу 6.4.

Даља измена статуса игре и брисање из библиотеке су могући на страници личне библиотеке и на страници са детаљима игре, а извршавају се позивима `POST /api/library` (као код уписа) и `DELETE /api/library/:gameId`. Заштићене руте захтевају пријаву на платформу, при чему се идентификатор корисника преузима из токена.

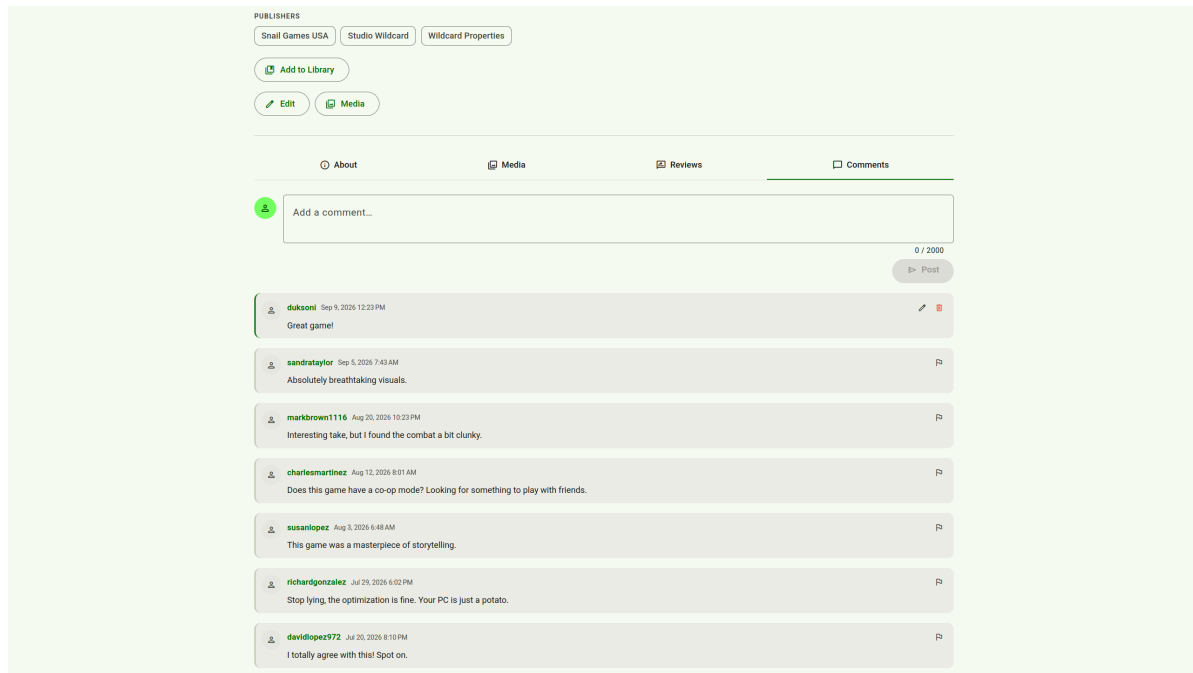
7.2.6 Оцењивање игара и коментарисање

На страници са детаљима игре постоје одвојени одељци са оценама и коментарима. Оцењивање је доступно пријављеним корисницима кликом на одговарајуће дугме у одељку са оценама и избором обавезне оцене уз опциони унос текста рецензије. Исти дијалог, приказан на слици 7, може да се отвори и на страници личне библиотеке кликом на одговарајуће дугме при преласку преко картице игре.



Слика 7: Страница личне библиотеке са отвореним дијалогом за оцењивање изабране игре.

Оцена се чува или ажурира позивом `POST /api/reviews`, а брише позивом `DELETE /api/reviews/:id`. Сервис за кориснички садржај пре уписа проверава постојање игре и идентитет аутора, како је описано у поглављу 6.5.



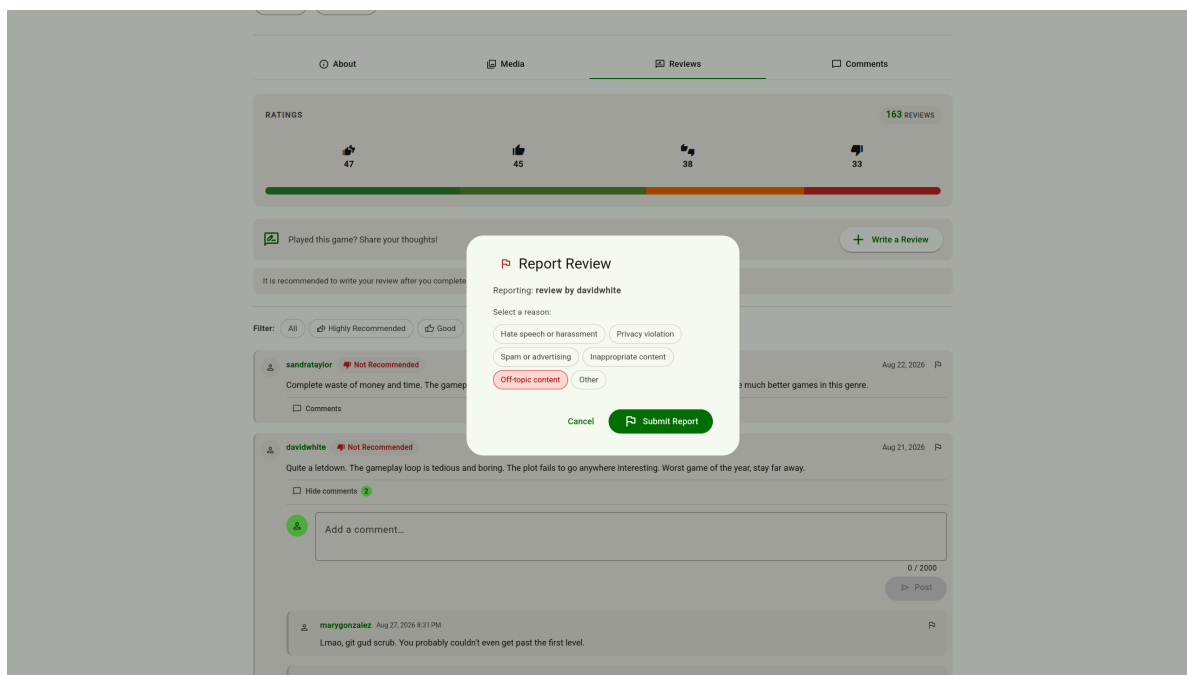
Слика 8: Одељак за коментаре странице са детаљима игре.

Оцене (са и без текста рецензије) могу да се коментаришу, а одељак за коментаре на страници са детаљима игре допушта коментарисање и саме игре, што је приказано на слици 8. Коментарисање је доступно пријављеним корисницима: коментар се уноси као текст и шаље кликом на одговарајуће дугме.

Коментари се чувају позивом `POST /api/comments`, мењају позивом `PUT /api/comments/:id`, а бришу позивом `DELETE /api/comments/:id`, при чему сервис за кориснички садржај проверава идентитет аутора.

7.2.7 Подношење пријава о непримереном садржају

У одељку са оценама или коментарима на страници са детаљима игре, пријављени корисник може да поднесе пријаву о непримереној рецензији или коментару кликом на симбол заставице, при чему ће се отворити дијалог приказан на слици 9. Могуће је изабрати предефинисан разлог пријаве или унети слободан текст.



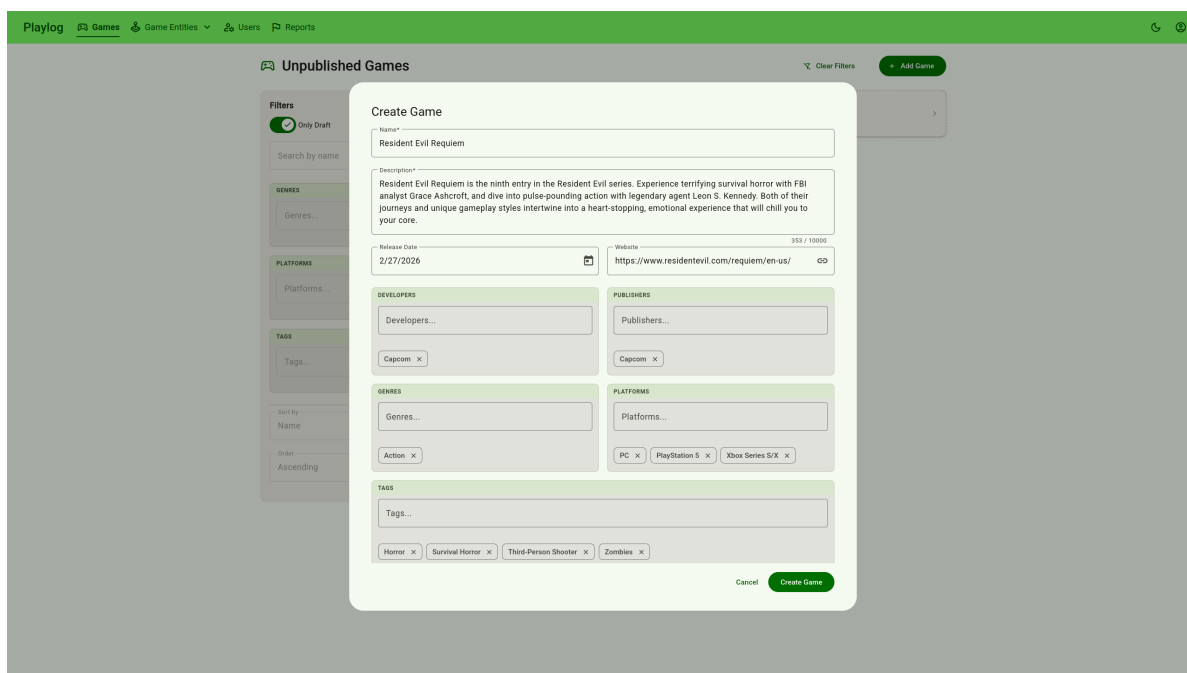
Слика 9: Страница са детаљима игре са отвореним дијалогом за подношење пријаве оцене.

Пријава се шаље позивом `POST /api/reports` са типом циљаног садржаја, идентификатором и образложењем. Сервис за кориснички садржај спречава дупле пријаве истог корисника за исти садржај.

7.3 Административне операције

7.3.1 Додавање игре у каталог

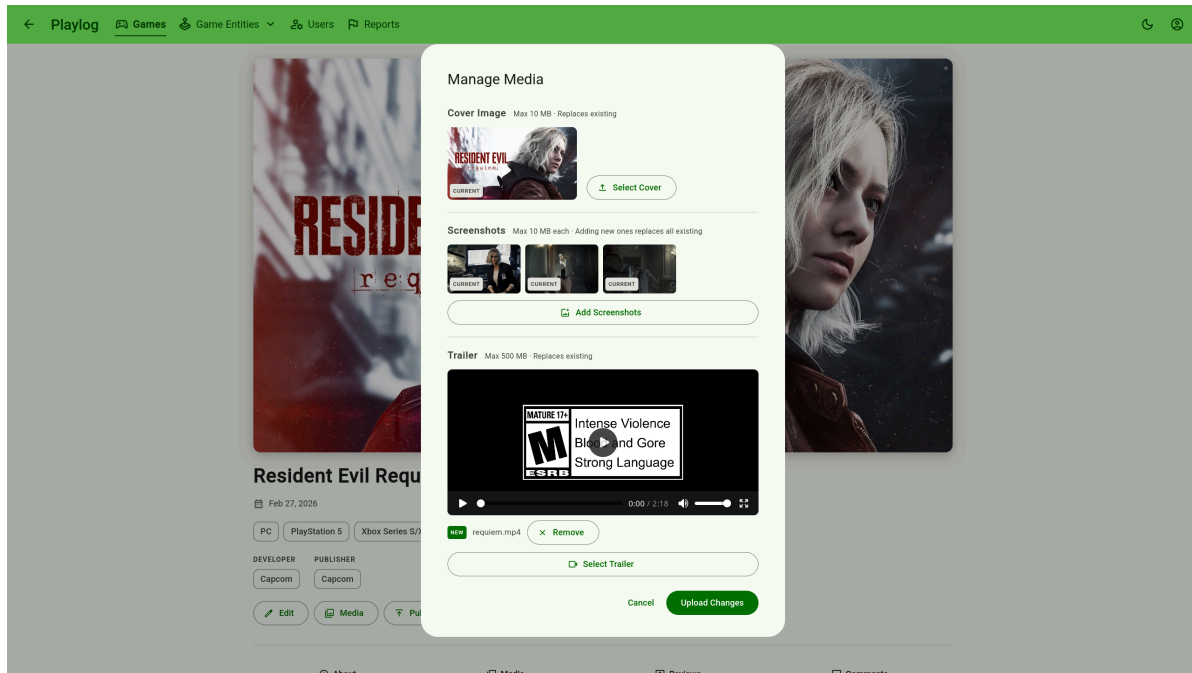
Са странице каталога, администратори могу да додају игру у каталог кликом на одговарајуће дугме, при чему ће се отворити дијалог приказан на слици 10. Уносе се основни подаци о игри и повезује се са постојећим пратећим ентитетима. Нова игра се чува као нацрт и видљива је само администраторима на страници са детаљима игре. Објавом игра постаје видљива свим корисницима. Подаци игре могу да се ажурирају у сваком тренутку, а игра може да се обрише само пре објаве (док је нацрт).



Слика 10: Додавање игре у каталог.

Упис се врши позивом `POST /api/games`, ажурирање позивом `PUT /api/games/:id`, објављивање позивом `PUT /api/games/:id/publish`, а брисање позивом `DELETE /api/games/:id`. Сервис за каталог пре уписа проверава постојање повезаних ентитета, а измене штити верзијом записа помоћу оптимистичног закључавања [17]. Подаци се чувају у табелама `games` и пратећим везама у бази.

На страници са детаљима игре, било да је игра објављена или у нацрту, администратори могу да уређују мултимедијални садржај кликом на одговарајуће дугме, при чему ће се отворити дијалог приказан на слици 11. Насловна слика, снимци екрана и трејлер се припремају одвојено, а шаљу заједничким захтевом.



Слика 11: Уређивање мултимедијалног садржаја игре.

Отпремање се врши позивом `POST /api/media/games/:gameId/upload` са верзијом записа и датотекама. Сервис за мултимедију најпре проверава постојање игре и исправност пристиглих датотека и верзије, затим складишти бајтове и метаподатке како је описано у поглављу 6.6.

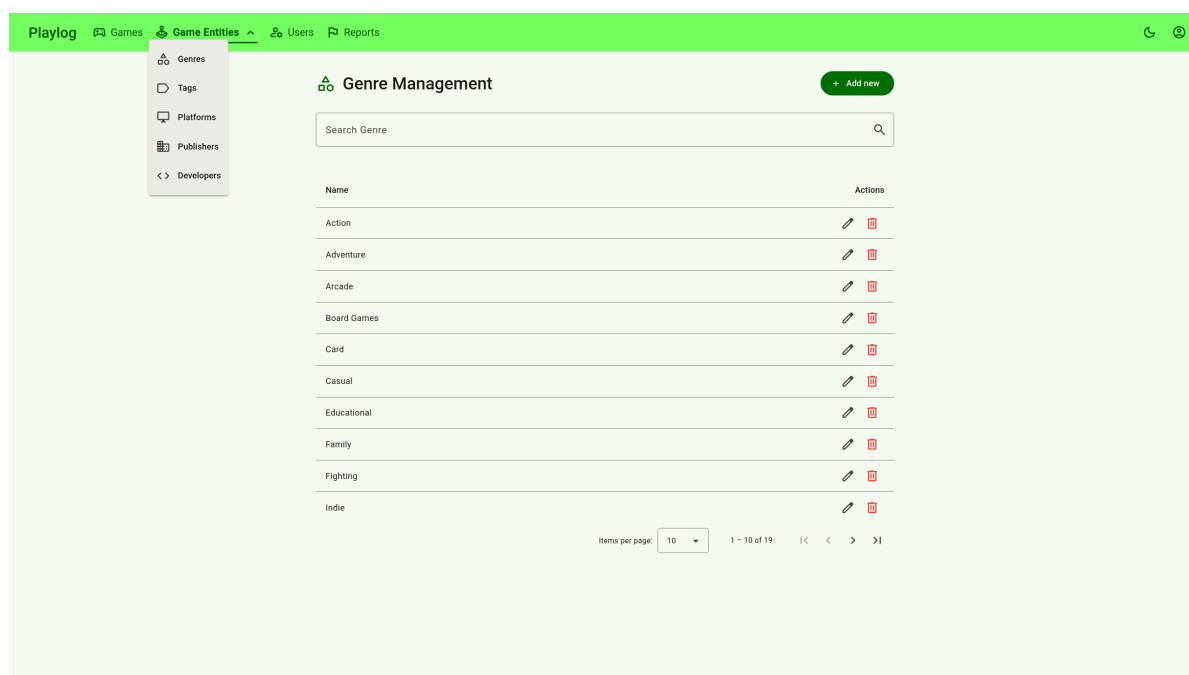
7.3.2 Управљање ентитетима везаних за игре

Из траке за навигацију, администратори могу да отворе мени са везама ка страницама за управљање пратећим ентитетима игара:

- програмерима,
- издавачима,
- жанровима,
- платформама и
- ознакама.

На слици 12 је конкретно приказана страница за управљање жанровима. За сваку врсту ентитета су омогућени претрага, приказ по странама, додавање и измена, док је брисање ограничено на жанрове, платформе и ознаке. Сами ентитети се састоје само од назива, па се додавање и измена своди на отварање дијалога кликом на одговарајуће дугме и унос новог назива или измене постојећег.

Операције се извршавају позивима `GET`, `POST`, `PUT` и `DELETE /api/:entity`. Сервис за каталог измене штити верзијом записа, како је описано у поглављу 6.3.

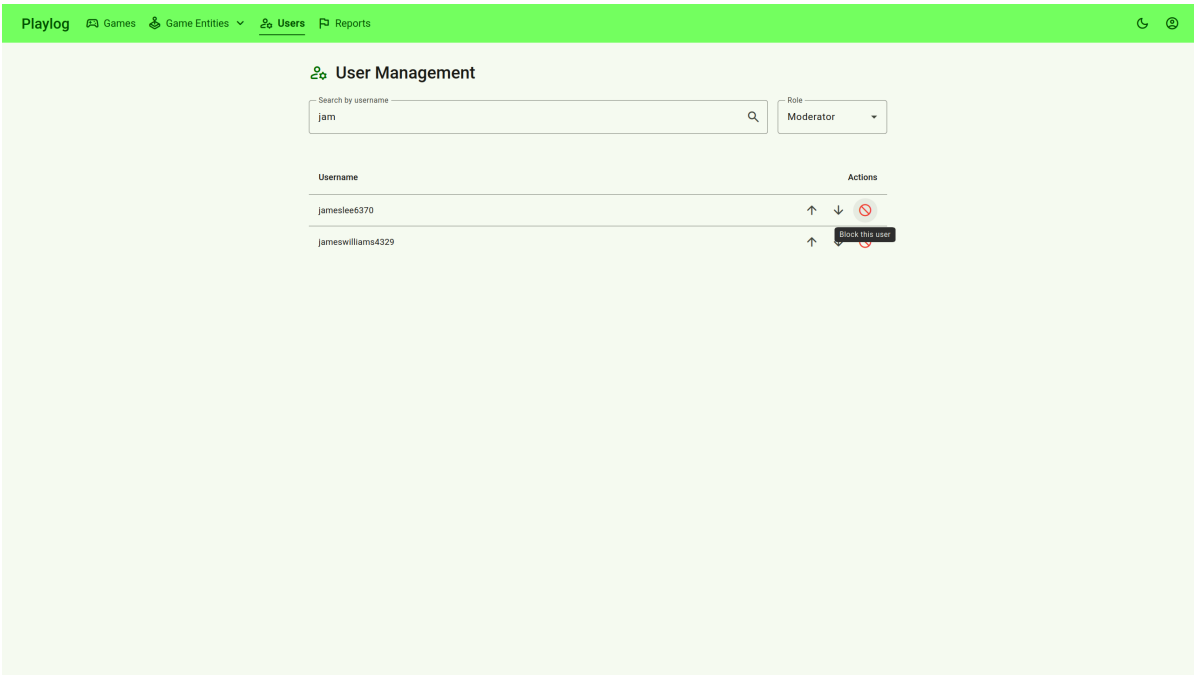


Слика 12: Страница за управљање жанровима. Исти распоред је и за остале ентитете.

7.3.3 Управљање налозима корисника

Страницу за управљање налозима корисника приказану на слици 13 администратори отварају из траке за навигацију. Кориснике могу да претражују по корисничком имену и да филтрирају по улози. Кликом на одговарајуће дугме појединачном кориснику могу да повећају или смање права приступа променом улоге, као и да му блокирају приступ налогу.

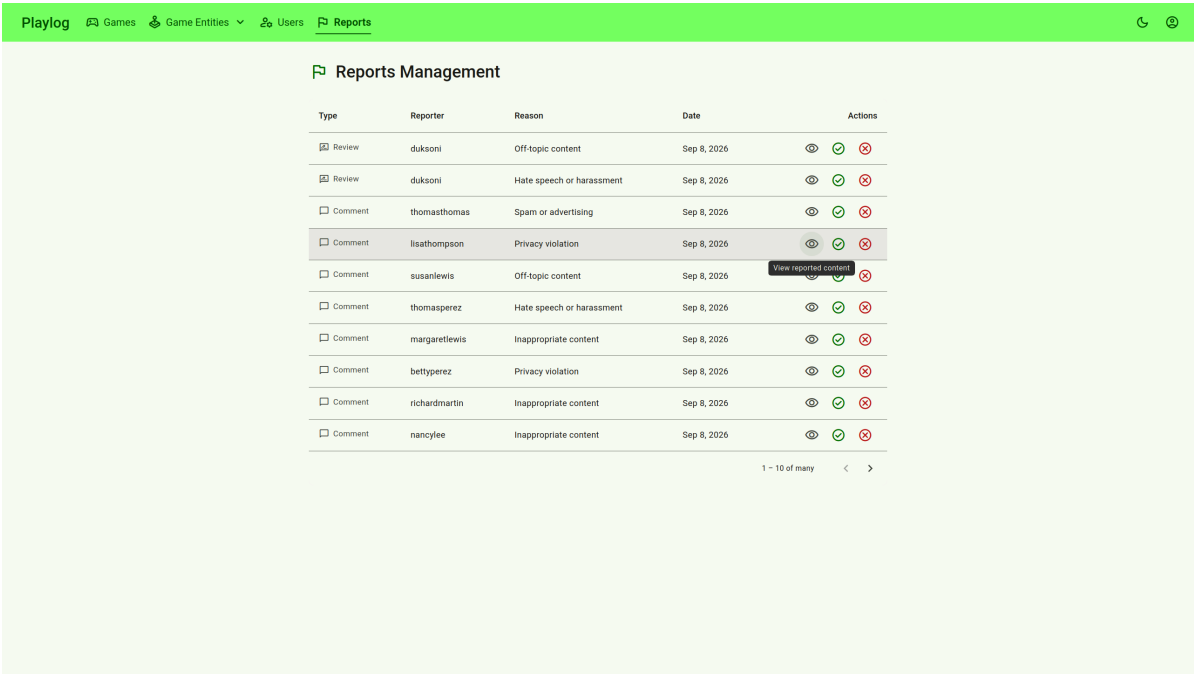
Претрага се врши позивом `GET /api/users?partial_username=:username&role=:role`. Промене улоге се врше позивима `PUT /api/users/:id/promote`, `PUT /api/users/:id/demote`, а блокирање `PUT /api/users/:id/block`. Сервис за кориснике измену улоге и блокирање спроводи у трансакцији над базом, уз повлачење издатих *refresh* токена блокираног или унапређеног налога.



Слика 13: Страница за управљање налозима корисника, са које је могуће блокирати кориснике или им променити улогу.

7.3.4 Модерација непримереног садржаја

Страницу пријављеног непримереног садржаја приказану на слици 14 модератори и администратори отварају из траке за навигацију, на којој могу да прегледају пристигле пријаве. Кликом на одговарајуће дугме могу да отворе дијалоге за проверу пријављеног садржаја, уважавање (логичко брисање пријављеног садржаја) или одбацивање пријаве. Уважена пријава скрива циљну рецензију или коментар. Модератори и администратори су спречени да доносе одлуке за пријаве које су лично поднели.



Слика 14: Страница за преглед пријављеног садржаја.

Добављање пријављеног садржаја се врши позивом GET /api/reports/pending, а разрешавање позивом PUT /api/reports/:id/status. Провера верзије при мењању статуса спречава двоструку обраду.

У раду је осмишљена и имплементирана микросервисна платформа за каталогизацију игара, управљање библиотеком игара и размену утисака. Систем чини шест сервиса приказаних у табели 3: *API gateway* и пет доменских сервиса организованих по обрасцу база података по сервису [9]. Складиштење обухвата три *PostgreSQL* базе, две *MongoDB* базе и *MinIO* објектно складиште. Серверски део је имплементиран у *Rust*-у помоћу сандука *axum*, а клијентски у *Angular*-у у комбинацији са *nginx*. Аутентификација се заснива на JWT токенима потписаним алгоритмом RS256 [10], који проверава *API gateway* и поново проверава сваки сервис према обрасцу *zero trust* [12]. Конкурентне уписе, изузев идемпотентног уписа у библиотеку, по правилу штити оптимистично закључавање [17]. Интерфејси сервиса су описани *OpenAPI* спецификацијом [16], а платформа се покреће преко *Docker Compose*-а [46] кроз *Makefile*.

У завршној фази израде платформе спроведена је неформална провера употребљивости са пет испитаника без програмерског искуства, кроз пет задатака, који обухватају претрагу игара, оцењивање игара, управљање налогом, управљање библиотеком и пријаву садржаја. Задатак је сматран успешним ако су га сви испитаници завршили без помоћи, а функционалност употребљивом ако је просечна субјективна оцена тежине (скала 1-5) износила највише 2,5. Сви задаци су завршени без помоћи, уз просечну оцену тежине један, чиме је услов испуњен за све посматране функционалности. Сви уочени недостаци корисничког интерфејса су убрзо отклоњени.

Главна предност изабраног решења је модуларност: свака функционалност је пројектована за независан развој, тестирање и пуштање у рад по сервисима, захваљујући подели по обрасцу база података по сервису [9] и организацији кода по моделу *package by feature* [18]. Типизирани грешке и упити проверени у време компајлирања додатно откривају велики део грешака пре покретања система. Интегритет података на нивоу шеме база чувају јединствена ограничења и сложени кључеви, а конкурентне измене штити верзија записа, тако да се сукоби пријављују уместо да се тихо препишу. Мултимедијални садржај се чита директно из објектног складишта преко унапред потписаних URL-ова, без прослеђивања бајтова кроз сервис за мултимедију.

Прототип ове платформе има своја ограничења. Пре свега, платформа се извршава на једном рачунару: плаћа се цена дистрибуиране архитектуре, без користи од скалирања на више инстанци. Провера постојања игре се врши синхроним позивом ка сервису за каталог при сваком упису у сервисе за библиотеку, кориснички садржај и мултимедију: застој или отказ каталога може да се пренесе на остале сервисе. Страница детаља игре се попуњава подацима добављеним паралелним позивима клијента ка више рута, што увећава број мрежних скокова. Недостају и устаљени механизми за рад у продукцији: кеш често читаних података, отпорност на отказ зависних сервиса и обједињено праћење захтева и метрика. Још један битан недостатак представља одсуство аутоматизованих тестова: постојећи тестови покривају само најкритичнију логику аутентификације на серверу (JWT и хеширање лозинки), па су се грешке у имплементацији сервиса откривале и отклањале тек накнадном ручном провером.

Даљи развој ка платформи отвореног кода спремној за продукцију зато обухвата пет праваца:

- **Смањивање спреге осталих сервиса са сервисом за каталог путем догађаја.** Сервис за каталог креирање, измену и брисање игара објављује преко трајног *message broker*-а, попут *RabbitMQ* [48], уз атомичност уписа и објаве и идемпотентне потрошаче догађаја. Остали сервиси при томе одржавају минималну локалну копију каталога и проверу спроводе локално, док синхрона провера остаје само резервни механизам при хладном покретању (енгл. *cold start*), са кратким роком чекања.

-
- **Јачање отпорности и перформанси**, након почетног испитивања оптерећења:
 - кратки рокови чекања одговора, подешени по свакој зависности, и ограничен број понављања захтева само за безбедна (идемпотентна) читања, уз образац *circuit breaker* [9],
 - *Redis* кеш [49] за читања каталога и збирне оцене, са инвалидацијом вођеном догађајима,
 - јединствена рута композиције API-ја (енгл. *API composition*) [9], која обједињује тренутне паралелне позиве за страницу детаља игре.
 - **Систематично тестирање**:
 - јединични тестови пословне логике и репозиторијума уз уграђени `#[test]` оквир (и `mockall` за симулацију зависности) на серверској страни,
 - интеграциони тестови рута слањем захтева директно ка рутеру (`tower::ServiceExt::oneshot`) [24],
 - јединични тестови компоненти алатом *Vitest* [50] на клијентској страни,
 - end-to-end провере корисничких токова алатом *Playwright* [51].
 - **Друштвене функционалности** као нови потрошачи истог тока догађаја, по узору на *Backloggd* и *RAWG* [6], [7]: најпре историја рецензија и коментара на профилу са приказом по странама, затим граф праћења са током активности и *push* обавештењима у апликацији, при чему поуздана достава *push* обавештења на телефон мотивише израду апликације изван оквира обичног прегледача. Прво би се увела прогресивна веб апликација са веб *push* обавештењима. Наменски клијент за телефон следи за њом, било као омотач око постојећег *Angular* кода (нпр. *Capacitor* [52]), било као нативна апликација. Међутим, нативна апликација захтева поновну израду целог корисничког интерфејса, па представља скупљу опцију.
 - **Припрема за продукцију**:
 - обједињено праћење захтева и метрика преко алата попут *OpenTelemetry* [53],
 - ограничавање броја захтева (*rate limiting*) на *API gateway*-у,
 - ојачан продукциони *Docker Compose* састав (шифрована веза, тајне, резервне копије),
 - *Kubernetes* [54] као пут даљег скалирања, када самостално хостовање преко *Docker Compose*-а више не буде довољно,

Списак слика

Слика 1	Архитектура платформе са током комуникације између компоненти. Главни ток је представљен пуним линијама.	16
Слика 2	Дијалози за регистрацију и пријаву на платформу.	34
Слика 3	Страница личног налога.	34
Слика 4	Претрага и филтрирање каталога.	35
Слика 5	Страница детаља игре са отвореним дијалогом за додавање игре у личну библиотеку.	36
Слика 6	Библиотека на страници јавног профила корисника.	36
Слика 7	Страница личне библиотеке са отвореним дијалогом за оцењивање изабране игре.	37
Слика 8	Одељак за коментаре странице са детаљима игре.	38
Слика 9	Страница са детаљима игре са отвореним дијалогом за подношење пријаве оцено.	39
Слика 10	Додавање игре у каталог.	39
Слика 11	Уређивање мултимедијалног садржаја игре.	40
Слика 12	Страница за управљање жанровима. Исти распоред је и за остале ентитете.	41
Слика 13	Страница за управљање налозима корисника, са које је могуће блокирати кориснике или им променити улогу.	42
Слика 14	Страница за преглед пријављеног садржаја.	42

Списак листинга

Листинг 1	Организација <i>Rust</i> пројекта	16
Листинг 2	Пример <i>package by feature</i> организације	17
Листинг 3	Комплетан склоп сервиса	17
Листинг 4	Заједнички редослед покретања (скраћено из <code>library-service/src/main.rs</code>)	18
Листинг 5	Провера токена и улога (<code>jwt-common/src/middleware.rs</code>)	19
Листинг 6	Заједничка грешка и алијас типа <code>Result</code> (<code>service-common/src/error.rs</code>)	20
Листинг 7	Хеширање лозинке алгоритмом <code>Argon2</code> (<code>user-service/src/shared/password.rs</code>)	20
Листинг 8	Исечак функције за генерисање пара токена алгоритмом <code>RS256</code> (скраћено из <code>user-service/src/auth/token.rs</code>)	21
Листинг 9	Функција за декодирање <code>JWT</code> токена (<code>jwt-common/src/token.rs</code>)	21
Листинг 10	Ротација <i>refresh</i> токена (<code>user-service/src/auth/service.rs</code>)	22
Листинг 11	Исечак индекса сервиса за каталог (<code>catalogue-service/migrations/20260218212643_init.up.sql</code>)	22
Листинг 12	Исечак <code>GameRepository trait</code> -а (скраћено из <code>catalogue-service/src/games/repository.rs</code>)	23
Листинг 13	Објављивање игре уз проверу верзије (скраћено из <code>catalogue-service/src/games/repository.rs</code>)	24
Листинг 14	Раздвајање јавних и администраторских рута (скраћено из <code>catalogue-service/src/games/handler.rs</code>)	24
Листинг 15	Упис или измена статуса једним упитом (<code>library-service/src/repository.rs</code>)	25
Листинг 16	Упис оцене уз проверу верзије (скраћено из <code>review-service/src/review/repository.rs</code>) ..	25
Листинг 17	Разрешавање пријаве (скраћено из <code>review-service/src/report/service.rs</code>)	26
Листинг 18	Отпремање са провером верзије и враћањем постављеног (скраћено из <code>multimedia-service/src/service.rs</code>)	27
Листинг 19	Стварање унапред потписаног <code>URL</code> -а ка објектном складишту (<code>multimedia-service/src/storage.rs</code>)	27
Листинг 20	универзални <i>proxy handler</i> за <i>endpoint</i> -ове свих сервиса (<code>api-gateway/src/proxy/handler.rs</code>)	28
Листинг 21	Прослеђивање захтева ка сервису (<code>api-gateway/src/proxy/client.rs</code>)	28
Листинг 22	Спајање спецификација сервиса (скраћено из <code>api-gateway/src/docs.rs</code>)	29
Листинг 23	Декларативни <i>OpenAPI</i> макро <i>handler</i> -а за регистрацију (исечак из <code>user-service/src/auth/handler.rs</code>)	29
Листинг 24	Освежавање и понављање захтева (скраћено из <code>app/core/interceptors/auth.interceptor.ts</code>)	30
Листинг 25	Заштита руте према улози (скраћено из <code>app/core/guards/auth.guard.ts</code>)	30
Листинг 26	Вишестепена слика серверског сервиса (скраћено из <code>user-service/Dockerfile</code>)	31
Листинг 27	Слика клијентске апликације (скраћено из <code>playlog-frontend/Dockerfile</code>)	31

Списак табела

Табела 1	Поређење монолитне и микросервисне архитектуре, према Ричардсону [9]	3
Табела 2	Приступ функционалностима по улогама	7
Табела 3	Микросервиси платформе	15
Табела 4	Изложени портови по окружењу	32
Табела 5	Обим припремљених демонстрационих података	33

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
DNS	Domain Name System (систем имена домена)
GIN	Generalized Inverted Index (уопштени инвертовани индекс)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
MIME	Multipurpose Internet Mail Extensions (вишенаменска проширења интернет поште)
OpenAPI	спецификација за описивање HTTP API-ја
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RSA	Rivest-Shamir-Adleman (асиметрични криптографски алгоритам)
RS256	RSA Signature with SHA-256 (RSA потпис са SHA-256)
S3	Simple Storage Service (једноставна услуга складиштења)
SHA	Secure Hash Algorithm (породица криптографских хеш функција)
SQL	Structured Query Language (структурирани упитни језик)
UI	User Interface (кориснички интерфејс)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
YAML	YAML Ain't Markup Language (формат за серијализацију података)

Списак коришћених појмова

Појам	Објашњење
Access токен	Краткотрајан JWT који се шаље уз сваки захтев и самостално проверава потписом. Носи скуп информација о кориснику и омогућава праћење сесије без чувања стања на серверу.
API gateway	Јединствена улазна тачка микросервисног система која рутира захтеве ка одговарајућим сервисима.
База података по сервису (енгл. <i>database per service</i>)	Образац по коме сваки микросервис има приватне трајне податке доступне само преко свог API-ја.
Bucket (S3)	Логичка јединица објектног складишта која групише објекте.
Docker Compose	Алат за заједнички опис и покретање више контејнера кроз <code>compose.yml</code> датотеку са сервисима, мрежама и волуменима.
Docker слика (енгл. <i>image</i>)	Непроменљив шаблон за покретање контејнера описан <code>Dockerfile</code> рецептом са корацима изградње.
Endpoint	Конкретна адреса HTTP API-ја са методом и путањом која излаже једну операцију сервера.
Handler	Функција за обраду захтева ка којој рутер усмерава захтев.
HttpOnly колачић	Колачић недоступан скриптама на клијенту, чиме штити садржај (нпр. <i>refresh</i> токен) од крађе путем скрипти на страни клијента.
Контејнеризација (енгл. <i>containerization</i>)	Паковање апликације са свим зависностима у изоловане контејнере који се једнако извршавају у сваком окружењу. Изолација се постиже на нивоу оперативног система.
Middleware	Компонента која обрађује захтев пре или после главне обраде.
Микросервисна архитектура	Стил у коме је систем скуп независно пуштаних сервиса груписаних око поддомена. Омогућава независан развој по сервису уз цену сложених дистрибуираних операција.
Објектно складиште (енгл. <i>object storage</i>)	Складиште које чува произвољне бинарне податке као објекте у <i>bucket</i> -има.
Оптимистично за-кључавање (енгл. <i>optimistic locking</i>)	Техника очувања интегритета код конкурентних измена провером верзије записа. Упис пролази само при поклапању верзије, иначе се пријављује конфликт.
Package by feature	Модел организације кода у коме су све датотеке једне функционалности у истом фолдеру. Структура одражава пословни домен уместо техничких слојева.
Presigned URL (унапред потписани URL)	Унапред потписана адреса која привремено одобрава директан приступ објекту без проласка кроз сервис.
Refresh токен	Дуготрајан JWT који служи само за издавање новог пара токена по истеку <i>access</i> токена.
Reverse proxy	Посредник између клијента и сервера који прослеђује захтев у име клијента и скрива унутрашњу структуру система.
Ротација токена (енгл. <i>token rotation</i>)	Поступак у коме се при свакој употреби <i>refresh</i> токена стари повлачи и издаје нов пар.

Сандук (енгл. <i>crate</i>)	Јединица компајлирања у <i>Rust</i> -у, бинарна или библиотечка.
<i>Trait</i>	<i>Rust</i> концепт за дефинисање скупа метода које тип мора да имплементира, слично интерфејсу у другим језицима.
<i>Zero trust</i>	Безбедносни образац по принципу никада не веруј, увек провери. Сваки захтев се проверава без обзира на порекло.

Биографија

Душан Комадиновић је рођен 11. септембра 2003. године у Сремској Митровици. Основну школу „Јован Јовановић Змај“ у Сремској Митровици је завршио 2018. године као носилац Вукове дипломе. Исте године је уписао билингвални смер у Митровачкој гимназији. Гимназију је завршио 2022. године као носилац Вукове дипломе. Исте године уписује основне академске студије на Факултету техничких наука Универзитета у Новом Саду, смер софтверско инжењерство и информационе технологије. Током друге и треће године студија, био је прималац студентске стипендије града Сремска Митровица, а у четвртој години прималац стипендије Доситеја Фонда за младе таленте Републике Србије. Положио је све предмете предвиђене планом и програмом са просечном оценом 9,73.

Литература

- [1] J. Kattilakoski, "The Growth of Video Game Industry: Current Status and Future Prospects," thesis, 2019. [Online]. Доступно на <https://urn.fi/URN:NBN:fi:amk-2019052411741>
- [2] Valve, "Steam Store." Accessed: August 27, 2026. [Online]. Доступно на <https://store.steampowered.com/>
- [3] GOG, "GOG.com Store." Accessed: August 27, 2026. [Online]. Доступно на <https://www.gog.com/en/>
- [4] V. Corporation, "Steam Community." Accessed: August 27, 2026. [Online]. Доступно на <https://steamcommunity.com/>
- [5] G. sp. z o.o., "GOG Forums." Accessed: August 27, 2026. [Online]. Доступно на <https://www.gog.com/forum>
- [6] "Backlogg'd - A Video Game Collection Tracker." Accessed: August 27, 2026. [Online]. Доступно на <https://backlogg'd.com/>
- [7] "The Biggest Video Game Database on RAWG." Accessed: August 27, 2026. [Online]. Доступно на <https://rawg.io/>
- [8] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc., 2021.
- [9] C. Richardson, "A pattern language for microservices." Accessed: August 27, 2026. [Online]. Доступно на <https://microservices.io/patterns/>
- [10] S. Peyrott, *The JWT Handbook*. Bellevue, WA, USA: Auth0 Inc., 2016.
- [11] I. Andrushchak and B. Kupyra, "Authentication and session management in modern web applications using JWT with rotating refresh tokens," *International Science Journal of Engineering & Agriculture*, 2026.
- [12] O. S. Adanigbo, B. I. Adekunle, E. Ogbuefi, O. T. Odofin, O. A. Agboola, and D. Kisina, "Implementing Zero Trust Security in Multi-Cloud Microservices Platforms: A Review and Architectural Framework," *International Journal of Advanced Multidisciplinary Research and Studies*, 2024.
- [13] B. B. Rad, H. J. Bhatti, and M. Ahmadi, "An introduction to docker and analysis of its performance," *International Journal of Computer Science and Network Security (IJCSNS)*, 2017.
- [14] D. Aivaliotis, *Mastering Nginx*. Packt Publishing Ltd, 2016.
- [15] MinIO, "MinIO Documentation." Accessed: August 27, 2026. [Online]. Доступно на <https://github.com/minio/docs>
- [16] O. Initiative, "OpenAPI Specification (3.2.0)." Accessed: August 27, 2026. [Online]. Доступно на <https://spec.openapis.org/oas/v3.2.0.html>
- [17] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [18] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall Press, 2017.
- [19] Steve Klabnik Carol Nichols and with contributions from the R. C. Chris Krycho, "The Rust Programming Language." Accessed: August 27, 2026. [Online]. Доступно на <https://doc.rust-lang.org/stable/book/>
- [20] P. G. D. Group, "PostgreSQL documentation." Accessed: September 4, 2026. [Online]. Доступно на <https://www.postgresql.org/docs/>

-
- [21] P. G. D. Group, “GIN indexes.” Accessed: September 4, 2026. [Online]. Доступно на <https://www.postgresql.org/docs/current/gin.html>
- [22] P. G. D. Group, “pg_trgm - trigram matching.” Accessed: September 4, 2026. [Online]. Доступно на <https://www.postgresql.org/docs/current/pgtrgm.html>
- [23] MongoDB, “MongoDB manual.” Accessed: September 4, 2026. [Online]. Доступно на <https://www.mongodb.com/docs/manual/>
- [24] “Axum - ergonomic and modular web framework.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/axum>
- [25] “Tokio - asynchronous runtime for Rust.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/tokio/>
- [26] “Tower-HTTP - HTTP-specific middleware and utilities.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/tower-http>
- [27] “Reqwest - HTTP client for Rust.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/reqwest>
- [28] “Serde - serialization framework for Rust.” Accessed: September 4, 2026. [Online]. Доступно на <https://serde.rs/>
- [29] “Validator - validation for Rust structs.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/validator>
- [30] “SQLx - async SQL toolkit for Rust.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/sqlx>
- [31] MongoDB, “MongoDB Rust driver documentation.” Accessed: September 4, 2026. [Online]. Доступно на <https://www.mongodb.com/docs/drivers/rust/>
- [32] MongoDB, “Aggregation operations.” Accessed: September 4, 2026. [Online]. Доступно на <https://www.mongodb.com/docs/manual/aggregation/>
- [33] “MinIO Rust SDK (minio-rs).” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/minio>
- [34] “Utoipa - compile time OpenAPI documentation for Rust.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.rs/utoipa>
- [35] “Utoipa-axum - axum bindings for utoipa.” Accessed: September 6, 2026. [Online]. Доступно на <https://docs.rs/utoipa-axum>
- [36] “Argon2 password hashing for Rust.” Accessed: September 6, 2026. [Online]. Доступно на <https://docs.rs/argon2>
- [37] OWASP, “Password Storage Cheat Sheet.” Accessed: September 4, 2026. [Online]. Доступно на https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [38] Google, “Angular documentation.” Accessed: September 4, 2026. [Online]. Доступно на <https://v22.angular.dev/>
- [39] Microsoft, “TypeScript documentation.” Accessed: September 4, 2026. [Online]. Доступно на <https://www.typescriptlang.org/docs/>
- [40] Google, “Angular Material documentation.” Accessed: September 4, 2026. [Online]. Доступно на <https://material.angular.dev/>

-
- [41] Bootstrap, “Bootstrap documentation.” Accessed: September 4, 2026. [Online]. Доступно на <https://getbootstrap.com/docs/>
- [42] Oven, “Bun documentation.” Accessed: September 4, 2026. [Online]. Доступно на <https://bun.sh/docs>
- [43] Docker, “What is a container?.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [44] Docker, “Dockerfile overview.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.docker.com/build/concepts/dockerfile/>
- [45] Docker, “Docker build cache.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.docker.com/build/cache/>
- [46] Docker, “Define services in Docker Compose.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.docker.com/reference/compose-file/services/>
- [47] Docker, “Use multiple Compose files.” Accessed: September 4, 2026. [Online]. Доступно на <https://docs.docker.com/compose/how-tos/multiple-compose-files/>
- [48] Broadcom, “RabbitMQ documentation.” Accessed: September 6, 2026. [Online]. Доступно на <https://www.rabbitmq.com/docs>
- [49] Redis, “Redis documentation.” Accessed: September 6, 2026. [Online]. Доступно на <https://redis.io/docs/latest/>
- [50] Vitest, “Vitest documentation.” Accessed: September 6, 2026. [Online]. Доступно на <https://vitest.dev/>
- [51] Microsoft, “Playwright documentation.” Accessed: September 6, 2026. [Online]. Доступно на <https://playwright.dev/>
- [52] Ionic, “Capacitor documentation.” Accessed: September 11, 2026. [Online]. Доступно на <https://capacitorjs.com/docs>
- [53] OpenTelemetry, “OpenTelemetry documentation.” Accessed: September 6, 2026. [Online]. Доступно на <https://opentelemetry.io/docs/>
- [54] C. N. C. Foundation, “Kubernetes documentation.” Accessed: September 6, 2026. [Online]. Доступно на <https://kubernetes.io/docs/>